FORKING: OVER-ENCODING CAUSES OVER-MEMORIZATION UNDER DATA REPLAY

Anonymous authors
Paper under double-blind review

ABSTRACT

This paper studies a generalization failure discovered during an automated architecture search for NanoGPT, which we call *forking*. When the training data is repeatedly consumed, models with an over-encoding n-gram memory branch exhibit a clear separation between training and validation loss, and this separation often grows in a staircase pattern aligned with epoch boundaries. We study this phenomenon in a controlled vanilla NanoGPT setting and reproduce it in a DeepSeek-style model with Engram. Mechanistically, we find that repeated updates sharpen the continuations observed in training and keep suppressing the probability of unseen continuations, so their loss grows pass after pass. The n-gram module partitions the model into weakly interacting context-specific subspaces, which makes this overfitting and sharpening especially pronounced. Low-frequency n-gram contexts contribute most of the gap, whereas larger training budgets and heavily crowded over-encoding tables suppress it, so forking goes unnoticed in conventional pretraining. These trends predict that short-budget, heavily repeated regimes such as SFT and RL should be prone to forking, which we verify even when the n-gram table is frozen.

1 INTRODUCTION

Language-model agents are increasingly used to propose, execute, and evaluate research ideas. Systems for automated scientific discovery and model improvement extend this process from individual coding tasks to iterative experimental search (Lu et al., 2024; Recursive, 2026). Such searches can improve a target metric, but they can also reveal behavior that was not part of the original objective. This paper studies one such discovery in a NanoGPT autoresearch workflow (Figure 1).

The agent's task was to modify the model architecture and trainer to minimize next-token validation bits per byte (BPB). During a small-data experiment, we observed an unexpected pattern: Training and validation losses initially improved together, But once the loader began replaying previously seen text, the curves separated rapidly near the epoch boundary. Training loss continued to fall, while validation loss stopped improving or even increased. The gap then grew in steps over subsequent passes. We call this replay-aligned divergence *forking* (Figure 2).

This behavior resembles overfitting: training loss continues to decrease while validation loss deteriorates, a pattern previously observed in language models trained on repeated data (Hernandez et al., 2022). However, such a sharp divergence is not an inevitable consequence of data reuse; previous studies show that language models can benefit from several passes over a finite corpus (Muennighoff et al., 2023). Through a series of controlled experiments, we trace forking to the interaction between data replay and *over-encoding*, which enriches token representations with additional embeddings of local token combinations (Huang et al., 2025). In our model, an n-gram module uses the most recent n tokens, including the current token, to retrieve a learned embedding from a hash-indexed table. This embedding is injected into the Transformer backbone to help predict the next token. Across the configurations we test, adding this module produces a pronounced training–validation gap once the training data are replayed.

To understand why over-encoding promotes overfitting under replay, we examine how the gap depends on context frequency. Low-frequency n-gram contexts account for a substantial share of the additional gap, and the average gap follows an approximate power-law relationship with context frequency over the measured range. We propose that the n-gram table makes optimization

more local by assigning distinct learned representations to different contexts. These representations allow the model to fit local prediction problems more independently, making it easier to memorize the continuations observed in training. For rare contexts, only a few valid continuations may be represented in the corpus. Replaying the same examples reinforces these continuations without expanding their coverage, potentially suppressing alternatives needed at validation time. We develop this explanation through frequency-dependent interventions, a model of continuation coverage, and an analysis of replay dynamics.

The rest of the paper is organized as follows. We begin in Section 2 by retracing the discovery of forking and establishing the phenomenon. With this observation in place, Section 3 examines the empirical patterns that shape it. Section 4 then develops a mechanistic account of those patterns. We turn in Section 5 to their relevance for post-training, before discussing future directions in Section 6. Due to space constraints, related work is discussed in Appendix A.

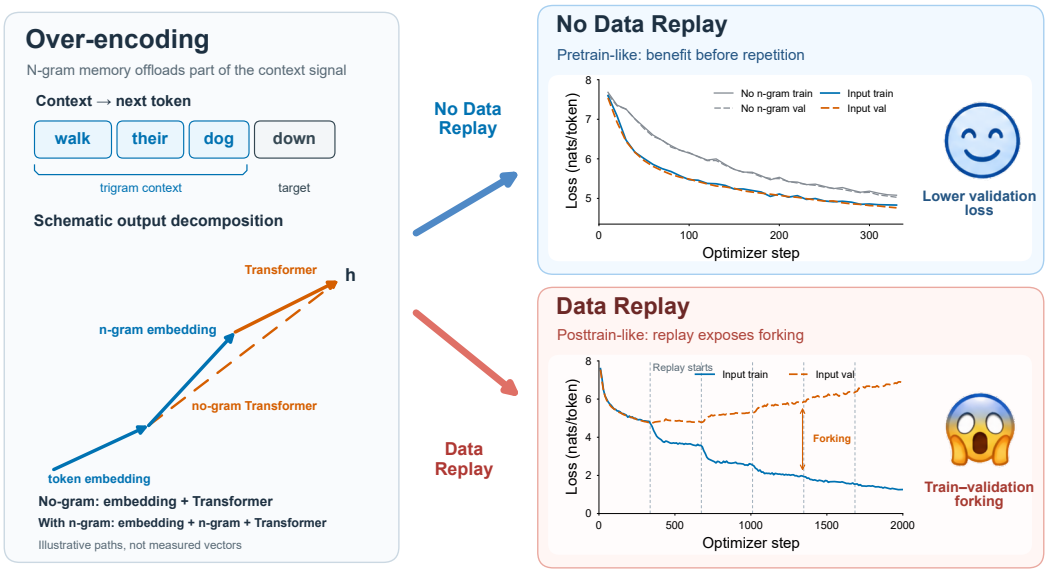


Figure 1: Over-encoding and replay-aligned forking. Left: an illustrative n-gram context and schematic contributions to the residual stream; arrow lengths and angles are not measured. Upper right: matched n-gram and no-gram runs during the first pass, before replay. Lower right: the n-gram run over repeated passes, with vertical lines marking pass boundaries and the first line marking the start of replay. Solid curves show online training loss and dashed curves show fixed-validation loss (nats/token). The upper panel is the first pass of a multi-epoch run, not a separately trained no-replay condition.

2 Discovery and Phenomenon

OPHIS.

We use a NanoGPT backbone with 8 layers, 6 attention heads, a hidden dimension of 768, a vocabulary of 8192 tokens, and a sequence length of 2048, together with separate bigram and trigram lookup tables. Retrieved embeddings are injected at one of three routes—token representations before attention (input), attention values (v), or attention output (y)—alongside a no-gram baseline (Figure 3); the input route is the default. The backbone uses AdamW with learning rate 6×10^{-4}; the tables use momentum-free RMSProp at 128 times the backbone learning rate. Learning rates are warmed up for 100 steps and then held constant. One epoch over the fixed single-shard training corpus takes 337 steps, and runs use seed 42 unless stated otherwise. Appendix C gives the full contract, numerical precision, and implementation details.

Training loss (L_{tr}) is the mean next-token cross-entropy on the current training batch, computed before the parameter update. Validation loss (L_{va}) is the mean next-token cross-entropy on fixed

held-out validation batches, evaluated after the update. At each evaluation step, the online gap is $G = L_{\text{va}} - L_{\text{tr}}$.

Figure 2 shows the signature pattern over 2000 updates, with dotted lines marking the 337-step replay boundaries. During the first epoch, training and validation losses improve together. Once the same text is replayed, training loss keeps falling without a corresponding validation improvement, and the gap widens—not smoothly but in a staircase aligned with the replay boundaries: most of the damage arrives right after each epoch transition and accumulates over successive passes. All three over-encoding routes fork; the attention routes (v, y) fork far more strongly than the input route, while the no-gram baseline stays small. The phenomenon is not specific to this minimal architecture: a 12-layer DeepSeek-style model with multi-layer Engram input over-encoding reproduces forking and responds to low-frequency read masking in the same way (Appendix G).

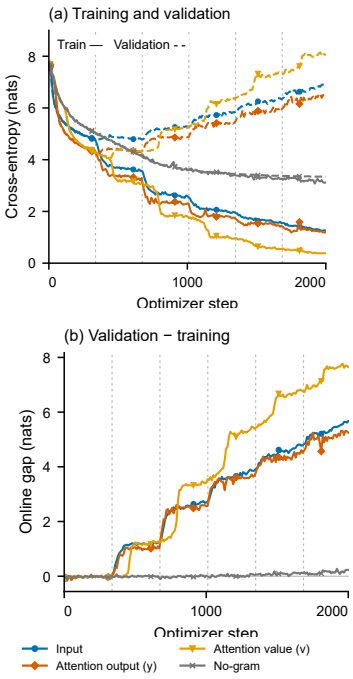


Figure 2: Forking under fixed data replay.

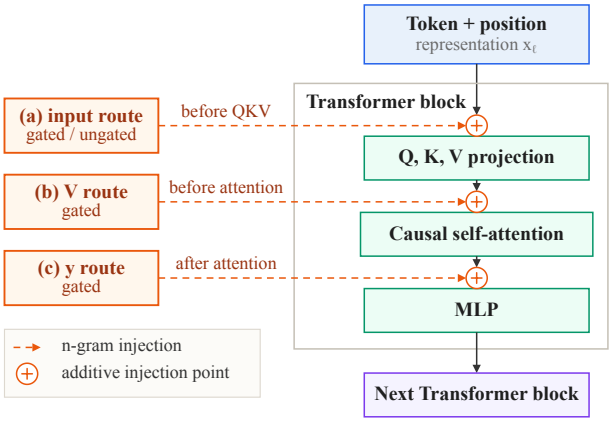


Figure 3: Three n-gram injection routes.

3 Empirical Laws and Interventions

3.1 Empirical scaling laws

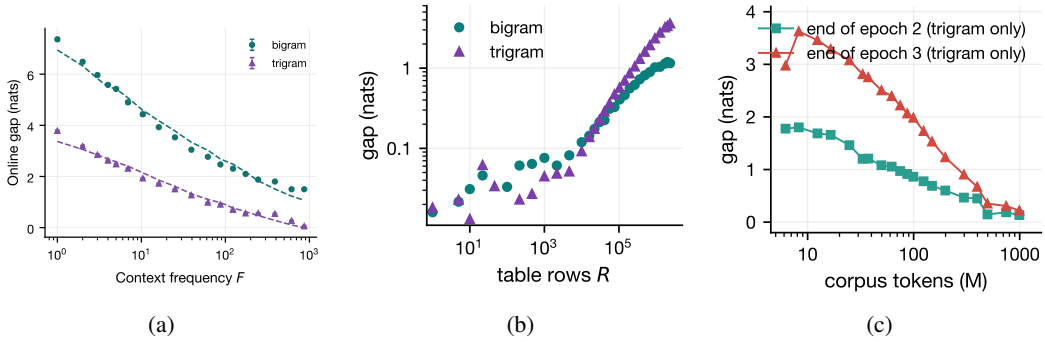
All experiments below use input injection and, where tables are enabled, a table learning rate 128 times the backbone learning rate. The frequency analysis uses seeds 42, 43 and 44; the remaining experiments use seed 42. All use the online gap of Section 2.

Context frequency. A finite corpus does not fully represent the continuation distribution of each context: a context whose population distribution assigns equal probability to C and D may appear three times with C and once with D in training, so fitting the sample underpredicts D on fresh data. Enabling both tables (2^{20} rows each) and measuring online gaps at step 1000, grouped by the context count F in the unrepeated corpus, shows that low-frequency contexts have systematically larger gaps (Figure 4a; markers are native-bin means, dashed lines the coverage-based fits of Appendix E.5).

Table capacity. Bigger tables make forking worse. Across three decades of table rows R in bigram-only and trigram-only models, the step-1000 gap generally increases, sublinearly on log-log axes, with the bigram curve flattening toward the upper end of the measured range (Figure 4b).

Training corpus size. Bigger corpora make forking milder. In a trigram-only model with 2^{20} rows trained for three epochs, varying the corpus over 20 sizes from 6M to 994M prediction tokens per epoch ($0.125D_1$–$20D_1$, where D_1 is the single-shard budget of 337 batches), the epoch-2 and epoch-3 end gaps fall by more than an order of magnitude as the corpus grows, reaching 0.13 and 0.22 nats at $20D_1$ (Figure 4c). Two factors can contribute to the decay: a context's unseen-continuation mass decreases with its training count (Appendix E.2), and, at fixed table size, a larger corpus spreads each table row over more distinct contexts, diluting per-context memorization. Both factors point the same way for production pretraining—a single pass over a corpus far larger than the memory—where the coverage account predicts weaker forking; the adverse corner, small corpora replayed for many passes, is exactly post-training, which we test in Section 5.

Epoch count. With the corpus fixed, replay alone drives the growth: over 20 epochs, epoch-end gaps increase monotonically for the trigram-only and both-table models while the no-gram control stays near zero (Figure 5b). Dense 2000-step traces (Figure 5a) show the growth concentrating at replay boundaries, matching Figure 2b.



(a) (b) (c)

Figure 4: Empirical scaling laws of the gap: (a) context frequency (both-table model, online gap at step 1000; native frequency-bin means over seeds 42, 43 and 44, with standard deviations; dashed curves use Equation (14) and directly measured novelty), (b) table capacity, (c) training corpus size.

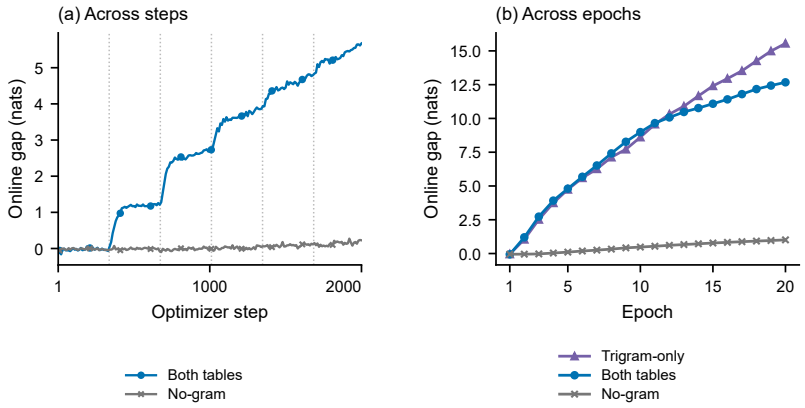


Figure 5: Gap growth under replay: (a) within epochs, (b) across 20 epochs.

3.2 Causal interventions

Two interventions in the both-table model test the frequency account directly. Both use the fixed single-shard training corpus, run for 1000 updates, and start the intervention at step 338, after the first epoch; the two panels of Figure 6 use independent runs, each with its own unmodified control.

Low-frequency retrieval masking. We count context frequencies in the unrepeated training corpus and mask the n-gram contribution whenever $F \leq \tau$, applying the rule separately to the bigram and trigram branches with thresholds $\tau \in \{1, 2, 4, 8, 200\}$. The retrieved embedding is set to zero during both training and evaluation, while the corresponding token remains in the training loss. Suppressing low-frequency memory access reduces the gap (Figure 6a): the excess gap is concentrated in rare contexts.

Freezing the tables. We train both n-gram tables normally during the first epoch, then freeze their parameters while continuing to update the backbone. A pronounced training–validation gap still develops after the tables are frozen (Figure 6b): once the tables hold learned context representations, the backbone alone can fit them during replay, so freezing the tables does not prevent forking.

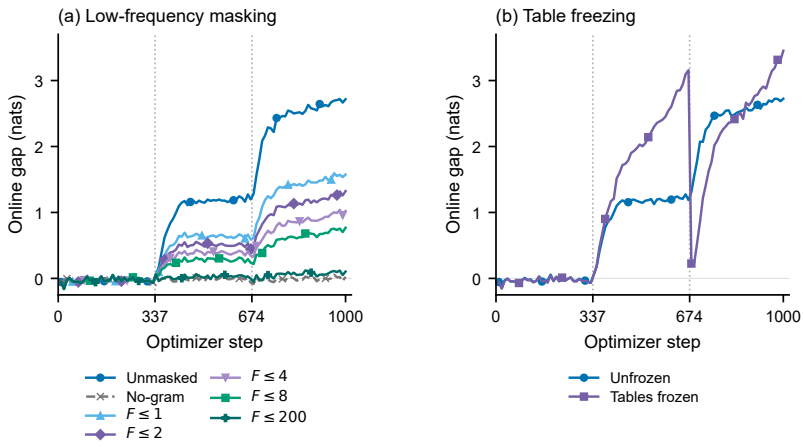


Figure 6: Causal interventions after epoch 1: (a) low-frequency retrieval masking, (b) table freezing.

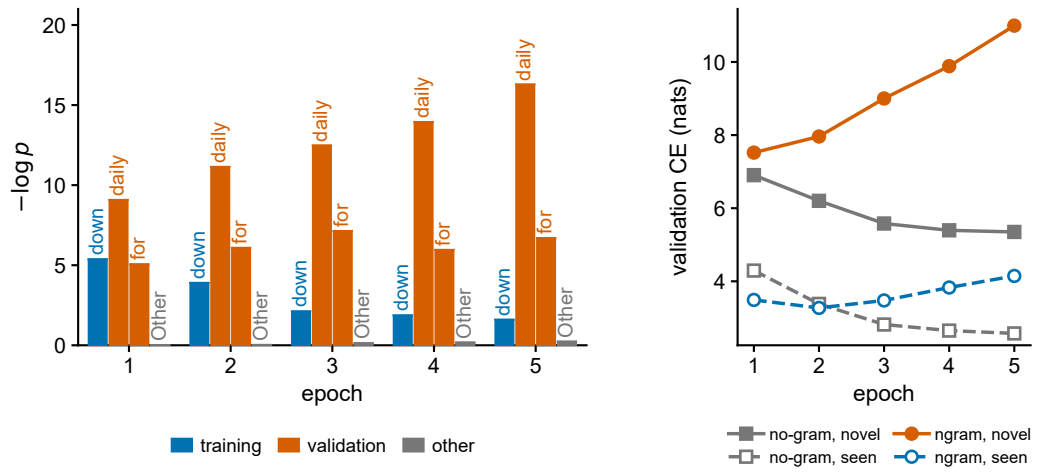
4 Mechanism of Forking

A finite corpus shows each context only some of its plausible continuations, so a validation position can pair a *seen context* with a *novel continuation* that never followed it in training. Replay reinforces the observed continuations without supplying evidence for the others, and can crowd those alternatives out. We show how this happens, then test why the n-gram module allows it to proceed separately within each context; additional experimental details are in Appendix F.

4.1 Crowding-out under replay

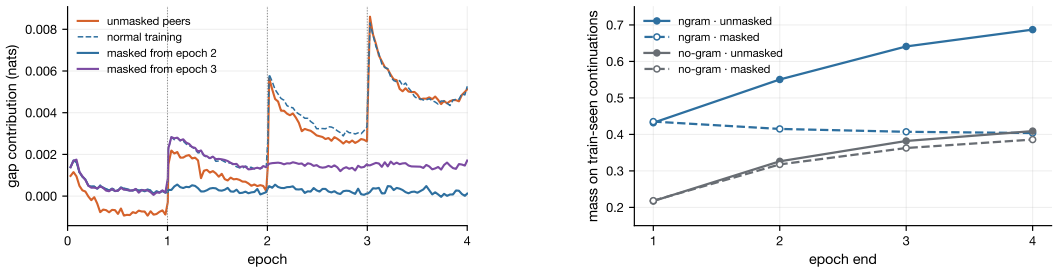
Forking is ordinary overfitting confined to very small problems. For a context c_i, call the next-token logits computed from it its *context subspace*, and split the vocabulary into the continuations of c_i seen in training, those that appear only in validation, and the rest (Appendix E.3). If training shows a single continuation C, the loss in that subspace is $-\ln p_C$; an idealized step directly in logit coordinates raises the logit of C by $\eta(1 - p_C)$ and, through the softmax, lowers every other logit by ηp_v. Mass flows from the unseen continuations onto the seen one, so the seen-to-unseen odds grow by a factor above one at every pass with nothing to stop them while $p_C < 1$ (Appendix Equation (22)). We call this *crowding-out*; it provides an idealized account of the growth in the novel-continuation penalty with replay.

Figure 7a shows the picture on one context. “walk their dog” is followed by *down* once in training and, in the fixed validation window, once each by *daily* and *for*. Probed with context-only input, the probability of *down* rises by more than an order of magnitude over five passes while *daily* and *for* are compressed—*daily* by orders of magnitude—and so is the total mass outside the continuations observed in either sample. Figure 7b shows that the context is typical. Pooling validation positions whose bigram context occurs in training, split by whether the continuation was seen there (405,058



(a) (b)

Figure 7: Crowding-out: (a) context-only continuation losses for "walk their dog"; (b) validation cross-entropy of train-seen and novel continuations, pooled over contexts.



(a) Gap contribution. (b) Mass on train-seen continuations.

Figure 8: Random context loss masking.

positions) or not (158,790), both models do well on seen continuations over the first three passes: the control's cross-entropy falls from 4.3 to 2.8 nats and the both-table model's stays near 3.4. On novel continuations they diverge: the control still improves, 6.9 to 5.6 nats, while the both-table model deteriorates from 7.5 to 9.0 with a step at every epoch boundary—the staircase of Figure 5. Consistently, at epoch 3 the both-table model's predictive entropy on validation is 2.93 nats against a cross-entropy of 5.26, whereas the control's are 3.59 and 3.58 (same runs as Figure 7b): far more confident than accurate, the signature of a model that has committed to few continuations per context (Carlsson et al., 2025). Crowding-out also explains why freezing the tables does not stop forking (Section 3.2): the subspace coordinate is already in place, and the backbone can continue the descent on its own.

Crowding-out hurts more where novel continuations are common. Rare contexts have had fewer chances to reveal their continuations; under heavy-tailed conditional distributions, the novel fraction falls slowly with frequency, offering an explanation for the approximate power-law trend in the gap. Appendix E develops this connection through the seen/novel gap decomposition and Good–Turing estimation, including the terms that prevent exact proportionality.

4.2 Context subspaces are independent

Section 4.1 treats each context subspace as evolving on its own, with the sharpening within it as the engine of forking. Both statements are claims about what the n-gram module buys: a private coordinate per context, so that updates driven by one context barely touch the conditional distribution of another. We test this by ablation. We hold out a random 2% of training trigram contexts and

exclude the next-token loss at their positions from the start of epoch 2 or epoch 3 onward, keeping inputs, trainable parameters and the loss denominator unchanged; a disjoint 2% serves as unmasked peers, and each masked run is paired with normal training from the same checkpoint. Figure 8a plots their gap contribution $C_S = \sum_{i \in S} q_i[\bar{\ell}_{\mathrm{val},i} - \bar{\ell}_{\mathrm{tr},i}]$ on fixed batches. Until the mask fires the masked curves track normal training; afterwards they freeze—for two and three further passes—while the unmasked and normal curves take a fresh jump at every replay boundary. The gap of a masked context stops growing the moment its own replay stops, although the remaining 98% of contexts keep training throughout: gap growth is generated inside each subspace, not imported from the others.

Figure 8b follows the sharpening coordinate itself, the predictive mass on the continuations of each context that training has seen. In the n-gram model the unmasked contexts pour mass into their seen continuations pass after pass ($0.43 \to 0.69$ by epoch 4) while the masked ones freeze ($0.43 \to 0.40$): once a context's own loss is removed, nothing else sharpens its subspace. Without the n-gram module the mask changes nothing ($0.22 \to 0.39$ masked vs. 0.41 unmasked), and replay raises the mass on seen and validation-only continuations alike ($0.013 \to 0.06$): every context's conditional distribution lives in parameters shared with all others, and repetition improves it holistically. The table therefore partitions optimization into weakly interacting context subspaces; inside a subspace, replay-driven sharpening proceeds unchecked by the continuations of other contexts, which is exactly the over-memorization assumed in Section 4.1. Masked contexts are not immutable—the shared backbone still drifts—but that drift moves train and validation losses together and never enters the gap of Figure 8a.

5 Forking in Post-Training

Production pretraining rarely replays data: corpora are large enough that each example is seen once, and Section 3.1 accordingly shows the gap shrinking as the training corpus grows. Post-training inverts both conditions. Supervised fine-tuning often replays small, expensive corpora for several epochs, and reinforcement-learning methods such as GRPO and PPO revisit the same prompts across rollout groups and optimization epochs. This section tests whether forking reappears under exactly these conditions.

Continued training on small replayed corpora. We start from the low-gap $20\times$-corpus trigram checkpoint (gap 0.214 at step 20,220), reset both optimizers with a fresh 100-step warmup, and replay three 333-batch corpora for five epochs: a same-distribution continuation of the pretraining corpus (ID), Magicoder (code), and OpenR1-Math (math). Each corpus is run with the standard trainable trigram table and with the table frozen while the backbone remains trainable, which is the common production choice. With a trainable table the gap reopens on every source: endpoint gaps are 2.463 (ID), 1.012 (Magicoder), and 0.726 (OpenR1) at seed 42 (Figure 9). Freezing the table substantially reduces but does not remove the effect: validation loss still ends 11% (ID) and 16% (Magicoder) above training loss, and 8% on OpenR1. The full protocol and endpoint table are in Appendix H.

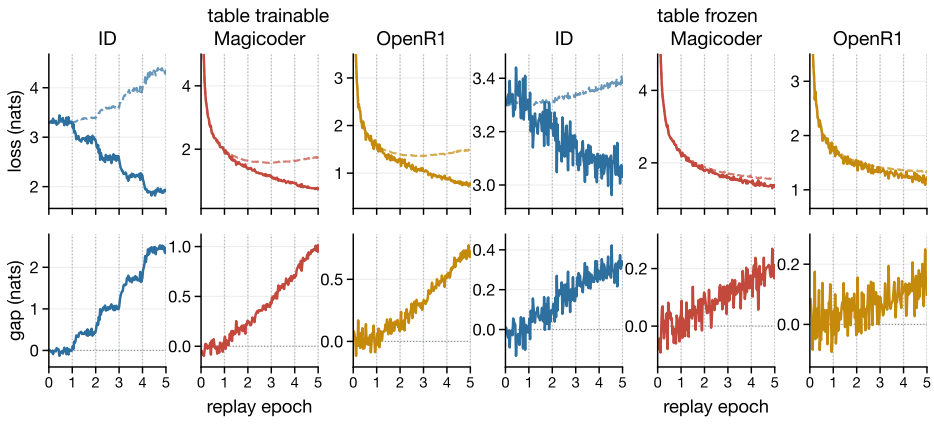


Figure 9: Continued-pretraining stress test: trainable trigram table (left), frozen table (right).

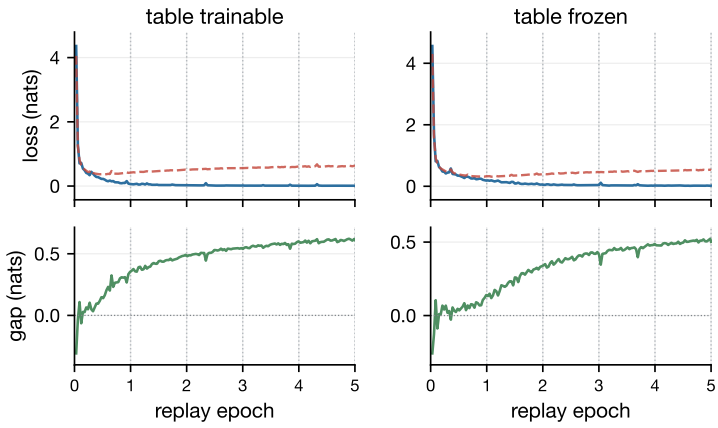


Figure 10: Static RLVR-like proxy on RLVR-MATH.

Verifier-selected data as an RL proxy. Verifier-filtered policy optimization concentrates training on accepted answers and repeats each prompt many times. As a static proxy for this regime (RFT or REINFORCE++-style data reuse, not on-policy RL), we replay RLVR-MATH from the Tulu 3 release (Lambert et al., 2025) with the reference answer appended once; each prompt recurs about 8.7 times per epoch, comparable to the per-prompt repetition of group-based sampling. The effect is stronger here than on the SFT corpora: endpoint gaps are 0.612 (trainable) and 0.502 (frozen), so even the frozen-table residual exceeds every SFT source (Figure 10).

Taken together, any post-training stage that replays a narrow corpus can reopen forking in a checkpoint that pretraining left nearly gap-free, and freezing the local memory mitigates but does not prevent it. Forking is therefore a failure mode that replay-heavy post-training pipelines must monitor, not an artifact of our minimal pretraining setting.

6 Future Work

Forking under over-encoding with data replay is a robust phenomenon, and its impact is largest in the short-budget, heavily repeated regimes of SFT and RL. This paper concentrates on the mechanism, which we consider reasonably complete; the next step is to characterize forking in production-scale models and to determine which production failures it accounts for. Such a study is currently limited by the scarcity of open over-encoding models—the most prominent, DeepSeek-V4.1-Flash (DeepSeek-AI, 2026), is very recent and very large—but its importance will grow as over-encoding matures.

Beyond loss regularities, future work should locate the downstream performance harm of forking and develop ways to avoid it. Masking low-frequency contexts is a promising candidate. An alternative is to restrict the over-encoding vocabulary at construction time by BPE-style aggregation, extrapolating the tokenizer-training algorithm so that low-frequency n-grams never enter the table; the over-encoding scheme of SCONE (Yu et al., 2025) already adopts this design.

Reproducibility Statement

Appendix C specifies the training contract used throughout: architecture, optimizers and learning-rate schedules, table sizes, data shards and epoch lengths, and the timing of the training and validation measurements. Every run uses a fixed data order and fixed validation batches, and each experiment states its seed. Appendix E gives the derivations behind the frequency law, and Appendix D the full intervention and replay curves. We will release the code, configurations, and training logs.

THE USE OF LARGE LANGUAGE MODELS

Large language models were used in three ways. First, forking was first observed during an automated architecture search in which a language-model agent proposed and evaluated modifications to a NanoGPT trainer (Section 2 and Appendix B). Second, language-model coding agents assisted the authors in implementing, launching, and analyzing experiments and in producing figures. Third, language models were used to edit text and to check bibliographic entries against their original sources. The authors directed all experiments, verified every reported number against the corresponding run logs and every reference against its source, and take full responsibility for the content of this paper.

REFERENCES

Vincent-Pierre Berges, Barlas Oguz, Daniel Haziza, Wen tau Yih, Luke Zettlemoyer, and Gargi Ghosh. Memory layers at scale. In Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu (eds.), *Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025*, volume 267 of *Proceedings of Machine Learning Research*, pp. 3831–3842. PMLR / OpenReview.net, 2025. URL https://proceedings.mlr.press/v267/berges25a.html.

Stella Biderman, USVSN Sai Prashanth, Lintang Sutawika, Hailey Schoelkopf, Quentin Anthony, Shivanshu Purohit, and Edward Raff. Emergent and predictable memorization in large language models. In Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine (eds.), *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023. URL http://papers.nips.cc/paper_files/paper/2023/hash/59404fb89d6194641c69ae99ecdf8f6d-Abstract-Conference.html.

Piotr Bojanowski, Edouard Grave, Armand Joulin, and Tomas Mikolov. Enriching word vectors with subword information. *Transactions of the Association for Computational Linguistics*, 5:135–146, 2017. doi: 10.1162/tacl_a_00051. URL https://aclanthology.org/Q17-1010/.

Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, Diego de Las Casas, Aurelia Guy, Jacob Menick, Roman Ring, Tom Hennigan, Saffron Huang, Loren Maggiore, Chris Jones, Albin Cassirer, Andy Brock, Michela Paganini, Geoffrey Irving, Oriol Vinyals, Simon Osindero, Karen Simonyan, Jack W. Rae, Erich Elsen, and Laurent Sifre. Improving language models by retrieving from trillions of tokens. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pp. 2206–2240. PMLR, 2022. URL https://proceedings.mlr.press/v162/borgeaud22a.html.

Nicholas Carlini, Daphne Ippolito, Matthew Jagielski, Katherine Lee, Florian Tramèr, and Chiyuan Zhang. Quantifying memorization across neural language models. In *International Conference on Learning Representations*, 2023. URL https://arxiv.org/abs/2202.07646.

Fredrik Carlsson, Fangyu Liu, Daniel Ward, Murathan Kurfali, and Joakim Nivre. The hyperfitting phenomenon: Sharpening and stabilizing LLMs for open-ended text generation. In *International Conference on Learning Representations*, 2025. URL https://arxiv.org/abs/2412.04318.

Stanley F. Chen and Joshua Goodman. An empirical study of smoothing techniques for language modeling. *Comput. Speech Lang.*, 13(4):359–393, 1999. doi: 10.1006/CSLA.1999.0128. URL https://doi.org/10.1006/CSLA.1999.0128.

Runxi Cheng, Yuchen Guan, Yongxian Wei, Qianpu Sun, Qixiu Li, Sinan Du, Feng Xiong, Chun Yuan, Yan Lu, and Yeyun Gong. Memory grafting: Scaling language model pre-training via offline conditional memory, 2026a. URL https://arxiv.org/abs/2605.20948.

Xin Cheng, Rui Tian, Wangding Zeng, Damai Dai, Qinyu Chen, Bingxuan Wang, Zhenda Xie, Kezhao Huang, Xingkai Yu, Chengqi Deng, Shangyan Zhou, Chenggang Zhao, Zhewen Hao, Yukun Li, Han Zhang, Zhengyan Zhang, Yixu Wei, M. Y. Xu, Huishuai Zhang, Dongyan Zhao, and Wenfeng

Liang. Conditional memory via scalable lookup: A new axis of sparsity for large language models. *arXiv preprint arXiv:2601.07372*, 2026b. URL https://arxiv.org/abs/2601.07372.

@classiclarryd. New record: Bigram hash embedding (-5.6s, -165 steps). modded-nanogpt speedrun record #62 (PR #201), 2026. URL https://github.com/KellerJordan/modded-nanogpt/tree/master/records/track_1_short/2026-01-19_BigramHashEmbedding.

DeepSeek-AI. DeepSeek-V4.1-Flash: Pushing the limits of KV cache compression. Model card and technical report, Hugging Face, 2026. URL https://huggingface.co/deepseek-ai/DeepSeek-V4.1-Flash. Released 2026-09-10.

Vitaly Feldman. Does learning require memorization? a short tale about a long tail. In Konstantin Makarychev, Yury Makarychev, Madhur Tulsiani, Gautam Kamath, and Julia Chuzhoy (eds.), *Proceedings of the 52nd Annual ACM SIGACT Symposium on Theory of Computing, STOC 2020, Chicago, IL, USA, June 22-26, 2020*, pp. 954–959. ACM, 2020. doi: 10.1145/3357713.3384290. URL https://doi.org/10.1145/3357713.3384290.

Vitaly Feldman and Chiyuan Zhang. What neural networks memorize and why: Discovering the long tail via influence estimation. In Hugo Larochelle, Marc'Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin (eds.), *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, 2020. URL https://proceedings.neurips.cc/paper/2020/hash/1e14bfe2714193e7af5abc64ecbd6b46-Abstract.html.

William A. Gale and Geoffrey Sampson. Good–turing frequency estimation without tears. *Journal of Quantitative Linguistics*, 2(3):217–237, 1995. doi: 10.1080/09296179508590051. URL https://www.grsampson.net/AGTf.pdf.

Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. Transformer feed-forward layers are key-value memories. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 5484–5495, 2021. doi: 10.18653/v1/2021.emnlp-main.446. URL https://doi.org/10.18653/v1/2021.emnlp-main.446.

Alexander Gnedin, Ben Hansen, and Jim Pitman. Notes on the occupancy problem with infinitely many boxes: General asymptotics and power laws. *Probability Surveys*, 4:146–171, 2007. doi: 10.1214/07-PS092. URL https://arxiv.org/abs/math/0701718.

I. J. Good. The population frequencies of species and the estimation of population parameters. *Biometrika*, 40(3–4):237–264, 1953. doi: 10.1093/biomet/40.3-4.237.

Edouard Grave, Armand Joulin, and Nicolas Usunier. Improving neural language models with a continuous cache. In *International Conference on Learning Representations*, 2017. URL https://openreview.net/forum?id=B184E5qee.

Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In Doina Precup and Yee Whye Teh (eds.), *Proceedings of the 34th International Conference on Machine Learning, ICML 2017, Sydney, NSW, Australia, 6-11 August 2017*, volume 70 of *Proceedings of Machine Learning Research*, pp. 1321–1330. PMLR, 2017. URL http://proceedings.mlr.press/v70/guo17a.html.

Danny Hernandez, Tom Brown, Tom Conerly, Nova DasSarma, Dawn Drain, Sheer El-Showk, Nelson Elhage, Zac Hatfield-Dodds, Tom Henighan, Tristan Hume, Scott Johnston, Ben Mann, Chris Olah, Catherine Olsson, Dario Amodei, Nicholas Joseph, Jared Kaplan, and Sam McCandlish. Scaling laws and interpretability of learning from repeated data. *arXiv preprint arXiv:2205.10487*, 2022. URL https://arxiv.org/abs/2205.10487.

Jeremy Howard and Jonathan Whitaker. Can LLMs learn from a single example? fast.ai blog, September 2023. URL https://www.fast.ai/posts/2023-09-04-learning-jumps/. Published September 4, 2023.

Hongzhi Huang, Defa Zhu, Banggu Wu, Yutao Zeng, Ya Wang, Qiyang Min, and Xun Zhou. Over-tokenized transformer: Vocabulary is generally worth scaling. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 26261–26282. PMLR, 2025. URL https://proceedings.mlr.press/v267/huang25bb.html.

Qian Huang, Jian Vora, Percy Liang, and Jure Leskovec. MLAgentBench: Evaluating language agents on machine learning experimentation. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 20271–20309. PMLR, 2024. URL https://proceedings.mlr.press/v235/huang24y.html.

Atindra Jha, Margaret Li, Jure Leskovec, Percy Liang, and Luke Zettlemoyer. Data scarcity and model sparsity: Mixtures-of-experts overfit more to repeated data. arXiv preprint arXiv:2609.11917, 2026. URL https://arxiv.org/abs/2609.11917.

Keller Jordan, Jeremy Bernstein, Brendan Rappazzo, @fernbear.bsky.social, Boza Vlado, You Jiacheng, Franz Cesista, Braden Koszarsky, and @Grad62304977. modded-nanogpt: Speedrunning the nanogpt baseline, 2024. URL https://github.com/KellerJordan/modded-nanogpt.

Nikhil Kandpal, Eric Wallace, and Colin Raffel. Deduplicating training data mitigates privacy risks in language models. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pp. 10697–10707. PMLR, 2022. URL https://proceedings.mlr.press/v162/kandpal22a.html.

Nikhil Kandpal, Haikang Deng, Adam Roberts, Eric Wallace, and Colin Raffel. Large language models struggle to learn long-tail knowledge. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett (eds.), *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA*, volume 202 of *Proceedings of Machine Learning Research*, pp. 15696–15707. PMLR, 2023. URL https://proceedings.mlr.press/v202/kandpal23a.html.

Andrej Karpathy. nanoGPT. GitHub repository, 2022. URL https://github.com/karpathy/nanoGPT. Accessed 2026-09-16.

Andrej Karpathy. Autoresearch. GitHub repository, 2026. URL https://github.com/karpathy/autoresearch. Accessed 2026-09-16.

Slava M. Katz. Estimation of probabilities from sparse data for the language model component of a speech recognizer. *IEEE Trans. Acoust. Speech Signal Process.*, 35(3):400–401, 1987. doi: 10.1109/TASSP.1987.1165125. URL https://doi.org/10.1109/TASSP.1987.1165125.

Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. Generalization through memorization: Nearest neighbor language models. In *International Conference on Learning Representations*, 2020. URL https://openreview.net/forum?id=HklBjCEKvH.

Reinhard Kneser and Hermann Ney. Improved backing-off for M-gram language modeling. In *1995 International Conference on Acoustics, Speech, and Signal Processing, ICASSP '95, Detroit, Michigan, USA, May 08-12, 1995*, pp. 181–184. IEEE Computer Society, 1995. doi: 10.1109/ICASSP.1995.479394. URL https://doi.org/10.1109/ICASSP.1995.479394.

Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James V. Miranda, Alisa Liu, Nouha Dziri, Shane Lyu, Yuling Gu, Saumya Malik, Victoria Graf, Jena D. Hwang, Jiangjiang Yang, Ronan Le Bras, Oyvind Tafjord, Chris Wilhelm, Luca Soldaini, Noah A. Smith, Yizhong Wang, Pradeep Dasigi, and Hannaneh Hajishirzi. Tulu 3: Pushing frontiers in open language model post-training. In *Second Conference on Language Modeling (COLM)*, 2025. URL https://openreview.net/forum?id=i1uGbfHHpH.

Guillaume Lample, Alexandre Sablayrolles, Marc'Aurelio Ranzato, Ludovic Denoyer, and Hervé Jégou. Large memory layers with product keys. In *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019. URL https://proceedings.neurips.cc/paper_files/paper/2019/file/9d8df73a3cfbf3c5b47bc9b50f214aff-Paper.pdf.

Katherine Lee, Daphne Ippolito, Andrew Nystrom, Chiyuan Zhang, Douglas Eck, Chris Callison-Burch, and Nicholas Carlini. Deduplicating training data makes language models better. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 8424–8445, 2022. doi: 10.18653/v1/2022.acl-long.577. URL https://doi.org/10.18653/v1/2022.acl-long.577.

Jiacheng Liu, Sewon Min, Luke Zettlemoyer, Yejin Choi, and Hannaneh Hajishirzi. Infini-gram: Scaling unbounded n-gram language models to a trillion tokens. In *First Conference on Language Modeling (COLM)*, 2024. URL https://openreview.net/forum?id=u2vAyMeLMm.

Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024. URL https://arxiv.org/abs/2408.06292.

Chris Lu, Cong Lu, Robert Tjarko Lange, Yutaro Yamada, Shengran Hu, Jakob Foerster, David Ha, and Jeff Clune. Towards end-to-end automation of AI research. *Nature*, 651(8107):914–919, 2026. doi: 10.1038/s41586-026-10265-5.

Niklas Muennighoff, Alexander M. Rush, Boaz Barak, Teven Le Scao, Aleksandra Piktus, Nouamane Tazi, Sampo Pyysalo, Thomas Wolf, and Colin Raffel. Scaling data-constrained language models. In *Advances in Neural Information Processing Systems*, volume 36, pp. 50358–50376. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/9d89448b63ce1e2e8dc7af72c984c196-Paper-Conference.pdf.

OpenAI. Model craft challenge: Parameter Golf. GitHub repository, 2026. URL https://github.com/openai/parameter-golf. Accessed 2026-09-22.

Artidoro Pagnoni, Ramakanth Pasunuru, Pedro Rodriguez, John Nguyen, Benjamin Muller, Margaret Li, Chunting Zhou, Lili Yu, Jason E. Weston, Luke Zettlemoyer, Gargi Ghosh, Mike Lewis, Ari Holtzman, and Srini Iyer. Byte latent transformer: Patches scale better than tokens. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 9238–9258, 2025. doi: 10.18653/v1/2025.acl-long.453. URL https://doi.org/10.18653/v1/2025.acl-long.453.

Recursive. First steps toward automated AI research. Blog post, June 2026. URL https://www.recursive.com/articles/first-steps-toward-automated-ai-research.

Aurko Roy, Rohan Anil, Guangda Lai, Benjamin Lee, Jeffrey Zhao, Shuyuan Zhang, Shibo Wang, Ye Zhang, Shen Wu, Rigel Swavely, Tao (Alex) Yu, Phuong Dao, Christopher Fifty, Zhifeng Chen, and Yonghui Wu. N-Grammer: Augmenting transformers with latent n-grams. *arXiv preprint arXiv:2207.06366*, 2022. URL https://arxiv.org/abs/2207.06366.

Raahil Shah. Int6 MLP3x + SmearGate + BigramHash + OrthoInit + Muon WD + SWA. OpenAI Parameter Golf record submission, 2026. URL https://github.com/openai/parameter-golf/tree/main/records/track_10min_16mb/2026-03-20_Int6_MLP3x_SmearGate_BigramHash_MuonWD_SWA. Record dated 2026-03-20; accessed 2026-09-22.

Dan Svenstrup, Jonas Hansen, and Ole Winther. Hash embeddings for efficient word representations. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/f0f6ba4b5e0000340312d33c212c3ae8-Paper.pdf.

Yee Whye Teh. A hierarchical Bayesian language model based on Pitman-Yor processes. In Nicoletta Calzolari, Claire Cardie, and Pierre Isabelle (eds.), *Proceedings of the 21st International Conference on Computational Linguistics and 44th Annual Meeting of the Association for Computational Linguistics*, pp. 985–992, Sydney, Australia, July 2006. Association for Computational Linguistics. doi: 10.3115/1220175.1220299. URL https://aclanthology.org/P06-1124/.

Kushal Tirumala, Aram H. Markosyan, Luke Zettlemoyer, and Armen Aghajanyan. Memorization without overfitting: Analyzing the training dynamics of large language models. In *Advances in Neural Information Processing Systems*, volume 35, pp. 38274–38290. Curran Associates, Inc., 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/fa0509f4dab6807e2cb465715bf2d249-Paper-Conference.pdf.

Kilian Q. Weinberger, Anirban Dasgupta, John Langford, Alexander J. Smola, and Josh Attenberg. Feature hashing for large scale multitask learning. In Andrea Pohoreckyj Danyluk, Léon Bottou, and Michael L. Littman (eds.), *Proceedings of the 26th Annual International Conference on Machine Learning, ICML 2009, Montreal, Quebec, Canada, June 14-18, 2009*, volume 382 of *ACM International Conference Proceeding Series*, pp. 1113–1120. ACM, 2009. doi: 10.1145/1553374.1553516. URL https://doi.org/10.1145/1553374.1553516.

Yuhuai Wu, Markus N. Rabe, DeLesley Hutchins, and Christian Szegedy. Memorizing transformers. In *International Conference on Learning Representations*, 2022. URL https://openreview.net/forum?id=TrjbxzRcnf-.

Frank F. Xu, Uri Alon, and Graham Neubig. Why do nearest neighbor language models work? In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 38325–38341. PMLR, 2023. URL https://proceedings.mlr.press/v202/xu23a.html.

Fuzhao Xue, Yao Fu, Wangchunshu Zhou, Zangwei Zheng, and Yang You. To repeat or not to repeat: Insights from scaling LLM under token-crisis. In Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine (eds.), *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023. URL http://papers.nips.cc/paper_files/paper/2023/hash/b9e472cd579c83e2f6aa3459f46aac28-Abstract-Conference.html.

Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. The AI scientist-v2: Workshop-level automated scientific discovery via agentic tree search, 2025. URL https://arxiv.org/abs/2504.08066.

Da Yu, Edith Cohen, Badih Ghazi, Yangsibo Huang, Pritish Kamath, Ravi Kumar, Daogao Liu, and Chiyuan Zhang. Scaling embedding layers in language models. In *Advances in Neural Information Processing Systems*, volume 38, pp. 31969–31995. Curran Associates, Inc., 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/file/2e067924aeeb02ae9919803fd08d8b4b-Paper-Conference.pdf.

Shanbin Yu, Danni Yu, Baiyu Zhu, Jinxin Wang, Peihang Li, Zirui Peng, Haoran Zhao, Leqian Yang, Yuting He, Shaoyang Guo, Zirui Ren, Qingyu Qu, Chencheng Tang, and Ziming Liu. OPHIS: Towards mechanistic auto-research. MetaCircle Blog, July 2026. URL https://meta-circle.com/blog/ophis-a-new-paradigm-for-autoresearch.

Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. Understanding deep learning requires rethinking generalization. In *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings*. OpenReview.net, 2017. URL https://openreview.net/forum?id=Sy8gdB9xx.

Chunting Zhou, Pengfei Liu, Puxin Xu, Srinivasan Iyer, Jiao Sun, Yuning Mao, Xuezhe Ma, Avia Efrat, Ping Yu, Lili Yu, Susan Zhang, Gargi Ghosh, Mike Lewis, Luke Zettlemoyer, and Omer Levy. LIMA: Less is more for alignment. In Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine (eds.), *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023. URL http://papers.nips.cc/paper_files/paper/2023/hash/ac662d74829e4407ce1d126477f4a03a-Abstract-Conference.html.

A Related Work

Hashed n-gram embeddings (over-encoding). Hashing maps an unbounded feature space into a fixed number of rows at the cost of collisions (Weinberger et al., 2009). FastText represents words by sums of character n-gram vectors (Bojanowski et al., 2017), and Hash Embeddings combine several shared hashed components with learned importance weights (Svenstrup et al., 2017). For language models, N-Grammer augments token representations with embeddings of latent bigram ids (Roy et al., 2022); the Over-Tokenized Transformer adds hashed embeddings of several n-gram orders to the input embedding and reports performance comparable to baselines about twice its size at no additional compute (Huang et al., 2025); SCONE learns contextual embeddings for a vocabulary of frequent n-grams and keeps them off-accelerator at inference (Yu et al., 2025); and BLT uses hashed byte n-gram embeddings when forming patch representations (Pagnoni et al., 2025). Engram retrieves hashed n-gram embeddings at several layers and gates them into the residual stream, outperforming an iso-parameter, iso-FLOPs mixture-of-experts baseline (Cheng et al., 2026b); an Engram memory with 196B parameters ships in the open-weight DeepSeek-V4.1-Flash (DeepSeek-AI, 2026). The same component has spread through speed and size competitions: a hashed bigram embedding is added to the residual stream in the modded-nanoGPT speedrun (Jordan et al., 2024; @classiclarryd, 2026), a BigramHash embedding appears in a Parameter Golf record (OpenAI, 2026; Shah, 2026), and an automated search for NanoChat training recipes arrived at hashed bigram and trigram embeddings injected into attention values (Recursive, 2026). All of these evaluations train on fresh data. We study the regime they leave out, repeated passes over a fixed corpus, and compare three injection routes.

Memory layers and retrieval. Product-key memories (Lample et al., 2019) and memory layers at scale (Berges et al., 2025) add large trainable key–value tables with sparse access, and feed-forward layers can themselves be read as key–value memories (Geva et al., 2021). Non-parametric alternatives interpolate with nearest neighbors in a datastore (Khandelwal et al., 2020; Xu et al., 2023), attend to retrieved chunks (Borgeaud et al., 2022), cache recent hidden states (Grave et al., 2017; Wu et al., 2022), or query unbounded n-gram counts (Liu et al., 2024). Memory Grafting precomputes values for frequent n-grams with a separate model and freezes them during pretraining (Cheng et al., 2026a). An n-gram table differs from learned-key memories in that its routing is fixed by the input tokens rather than learned; we argue that this fixed routing, which gives every context its own coordinates, is what lets repetition turn into per-context memorization (Section 4.1).

Repeated data and multi-epoch training. Under a fixed data budget, up to about four epochs of repetition are nearly as useful as fresh data, with rapidly diminishing returns afterwards (Muennighoff et al., 2023); a related study finds that repeated pre-training data causes multi-epoch degradation, that dataset size, model size, and training objective are the main factors, and that dropout is the only common regularizer that helps substantially (Xue et al., 2023). Repeating a small subset of the data can disproportionately harm test loss and damage copying and induction-head circuits (Hernandez et al., 2022). Duplication also increases verbatim extraction, which deduplication reduces (Carlini et al., 2023; Lee et al., 2022; Kandpal et al., 2022). Multi-epoch fine-tuning is routine in post-training—LIMA fine-tunes on 1,000 examples for 15 epochs (Zhou et al., 2023)—and practitioners fine-tuning an LLM have observed training loss jumping down at every epoch boundary, which they attributed to memorization after a single exposure (Howard & Whitaker, 2023). Hyperfitting deliberately overfits a small set until near-zero training loss and reports worse validation loss but better long-form generation (Carlsson et al., 2025). Concurrently with our work, Jha et al. (2026) find that mixture-of-experts models overfit repeated data more, and earlier, than dense models, that the effect grows with sparsity as set by total rather than active parameters, and that routing stabilizes early in training while expert specialization correlates with overfitting. An n-gram table can be viewed as the extreme case of this sparsity: its routing is fixed at step zero and each row receives exactly as many updates as its context occurs. Our contribution is a frequency-resolved law for the resulting gap, together with causal interventions and a mechanism.

Memorization and the long tail. Deep networks can fit random labels (Zhang et al., 2017), and memorizing rare examples can be necessary for near-optimal generalization when the data distribution is long-tailed (Feldman, 2020; Feldman & Zhang, 2020). In language models, larger models memorize faster and forget less (Tirumala et al., 2022), memorization of specific sequences is hard to predict from smaller runs (Biderman et al., 2023), and knowledge about rare entities is

learned poorly (Kandpal et al., 2023). Overconfidence of modern networks relative to their accuracy is a known failure (Guo et al., 2017); the n-gram model shows the same signature under replay, with a predictive entropy far below its cross-entropy on held-out continuations (Section 4). We locate a specific architectural source of long-tail memorization, the per-context coordinates of an over-encoding table, and show that rare contexts carry most of the resulting gap.

Missing mass and smoothing. Good–Turing estimation infers the probability of unseen outcomes from singleton counts (Good, 1953; Gale & Sampson, 1995), and occupancy theory relates the decay of this missing mass to the tail of the sampling distribution (Gnedin et al., 2007). Classical n-gram language models needed smoothing precisely because maximum-likelihood estimates assign zero probability to unseen continuations: Katz back-off, Kneser–Ney smoothing, and hierarchical Pitman–Yor models reserve mass for them in different ways (Katz, 1987; Kneser & Ney, 1995; Chen & Goodman, 1999; Teh, 2006). Our analysis suggests that a trainable n-gram table replayed inside a Transformer recreates the unsmoothed failure: repetition pushes each context toward its empirical continuation distribution, and low-frequency masking can be read as a crude back-off.

Automated research. Language-model agents have been benchmarked on machine-learning experimentation (Huang et al., 2024) and used to automate larger parts of the research cycle, from idea generation to experiments and paper writing (Lu et al., 2024; Yamada et al., 2025; Lu et al., 2026). Automated searches over training code target fixed-budget metrics, such as validation bits per byte, for small language models built on NanoGPT and nanochat (Karpathy, 2022; 2026; Recursive, 2026). OPHIS guides such searches with measurements of internal training dynamics (Yu et al., 2026); forking was first observed during an OPHIS-guided architecture search on NanoGPT. Our work illustrates a by-product of automated search: a component selected for a better target metric under a single-pass budget can fail under a different data regime, and the failure is only visible when the regime changes.

B Original Discovery and Observable Screening

B.1 The original task and discovery setting

NanoGPT RSI searches for changes that reduce validation bits per byte (BPB). An agent modifies `train.py` within a fixed compute budget, evaluates the result, and continues to revise the architecture and training strategy. The original automated search used a training budget of approximately five minutes per experiment. The small-data experiment in which forking was discovered instead used 2000 optimizer steps to inspect a longer trajectory. Forking was an unexpected outcome of this process.

The text came from the Hugging Face dataset `karpathy/climbmix-400b-shuffle`, using shards 1–10 and 6542. In the discovery-stage split, shards 1–3 were used for training, and shards 4–10 and 6542 for validation. This was a stage-specific small-data configuration, not the default split for the entire automated search. The loader read and packed text in a fixed order, then restarted after traversing the training shards. Three shards did not provide enough unique data for 2000 steps, so the run necessarily entered later epochs. Training and validation losses initially decreased together. When text began to repeat, training loss continued to fall while validation loss flattened or increased, producing a rapid divergence near the epoch boundary.

The agent-added n-gram module used independent learnable lookup tables at different layers. Each bigram or trigram context was mapped through two hashes. The two retrieved half-width vectors were concatenated, then injected into the attention value branch through an input-dependent sigmoid gate:

$$v_t^{(\ell)} \leftarrow v_t^{(\ell)} + g_{2,t}^{(\ell)} \odot e_{2,t}^{(\ell)} + g_{3,t}^{(\ell)} \odot e_{3,t}^{(\ell)}. \tag{1}$$

Here $e_{n,t}^{(\ell)}$ is the concatenated lookup result; the gate is computed from the corresponding channels of that layer's input. Hash constants and table parameters differ across layers. The retrieved vectors enter the value branch, rather than being added directly to the token embedding.

Table 1 records the frozen configuration used in the observable-localization experiments below. It differs from the later controlled setting in Section 2.

Table 1: Configuration for discovery-stage observable screening. Layer indices start at one.

Component	Configuration
Backbone	8 layers; hidden dimension 768; 6 attention heads; head dimension 128.
Input and batch	Sequence length 2048; 72 sequences or 147456 tokens per update; gradient accumulation 1.
Tokenizer	Shared `tokenizer_fixed_tr10`; vocabulary 8192; built from the first ten shards.
N-gram tables	Two tables per order and injection layer; each has 524288 rows of dimension 384. Bigram at layers 2, 4, 6, 8; trigram at layers 2, 6, 8.
Optimizers	Muon for matrix parameters, base learning rate 0.04; grouped AdamW for token/value embeddings, output and scalar parameters; RMSProp for bigram/trigram tables, learning rate 0.6, second-moment decay 0.999, no weight decay.
Schedule	Archived baseline schedule: Muon and ordinary AdamW warmdown begins at 5% and 35% of the step budget, respectively; final multiplier 0.05. N-gram table learning rates remain constant.
Logging	Four runs, each 1000 steps, seed 42. Validation uses four batches every five steps. Optimizer observables are normally logged every five steps and at every step in the 320–380 window, alongside parameter and layer statistics.

B.2 Localizing the n-gram pathway across controlled replay schedules

In the archived `20260714_opobs_tr1_1000step_seed42` run, we overlaid three series: `loss.train`, `val_loss`, and `rmsprop.update.group.ngram_embeddings.l1`. The L1 update magnitude is divided by 10^8 on the right axis; the losses use the left axis. The plot joins the recorded values without smoothing. Validation was evaluated every five updates and the last recorded value was carried forward between evaluations; optimizer observables were logged every five steps except for dense logging around the first replay transition. The exact pass boundaries at steps 338 and 687 come from the training log, rather than from the more sparsely sampled epoch observable. This one run is only a diagnostic view; the controls and screening procedure below establish why the component was selected for inspection.

B.2.1 Four experiments used to screen observables

We varied when training data were first reused and how reuse was organized, while fixing the model, optimizer configuration, tokenizer, and step budget. The four experiments distinguish ordinary replay, delayed replay, random repetition without a full-pass transition, and an imposed replay cycle.

TR1: sequential replay of one shard. The loader repeatedly traversed shard 1 in a fixed order. The second epoch started at step 338, followed by pronounced forking; the third started at step 687, with a further increase in the gap. This run supplied the boundary event to be localized.

TR2: sequential replay of two shards. The loader used shards 1 and 2 before restarting. At approximately step 350, it was still in the first traversal, and the training and validation curves remained close. The second epoch started at step 683, and the divergence was delayed accordingly. This control distinguishes a response to reuse from a change tied only to the absolute training step.

Random sampling with replacement. Documents were sampled uniformly with replacement from shard 1, then tokenized and packed into batches. Repeated documents could occur from the beginning of training. A gap developed gradually, but there was no full-pass replay transition near step 350. This supplied a control for the local boundary event.

50-new / 50-replay. The loader read 50 consecutive new batches from the shard-1 stream, then replayed those cached batches in the same order. Training loss fell and validation loss rose during replay; the curves moved closer when new data resumed. The resulting cycle of approximately 100 steps provided a test of whether an observable followed the replay state. Near the end of a base-stream epoch, a short block was replayed only for its actual cached length, so the local period could be shorter than 100 steps.

TR1, replacement sampling, and 50/50 used validation shards 2–10 and 6542. TR2 used shards 3–10 and 6542. These validation sets were therefore not identical. Screening compared the temporal responses of internal observables; loss curves identified the event within each run. In this appendix, a step counts completed optimizer updates. The centers 350 and 700 refer to screening windows, while actual epoch boundaries come from loader logs.

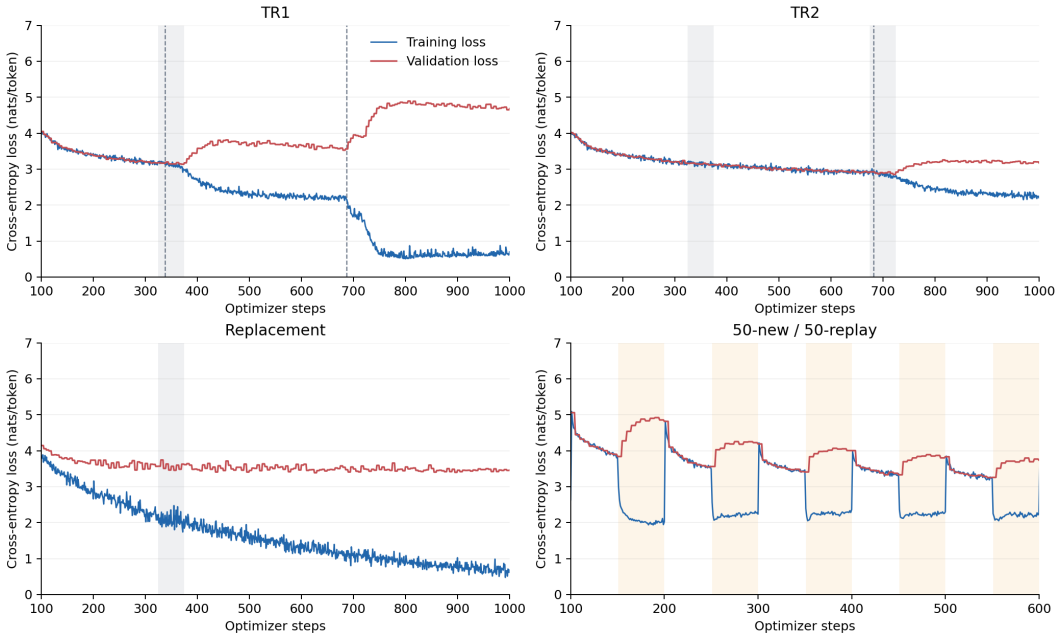


Figure 11: Four data-reuse controls. Blue and red curves show training and validation cross-entropy. Gray shading marks 350 ± 25 and, for TR2, 700 ± 25; dashed lines mark logged epoch boundaries. The 50/50 panel shows steps 100–600, with replay intervals in orange; the other panels show steps 100–1000. Archived curves are unsmoothed. Between validation evaluations, the most recent value is carried forward for display.

B.2.2 Five observable-screening rules

OPHIS records statistics of parameters, gradients, optimizer states, and update vectors, including their directional relationships (Yu et al., 2026). Candidates were scalar time series shared by the four runs. We excluded bookkeeping quantities such as data progress, learning-rate multipliers, throughput, and parameter counts, as well as some auxiliary fields used to construct other statistics. Window calculations used finite recorded values without interpolation.

Let $\mu(X)$ and $\sigma(X)$ denote the mean and population standard deviation of the recorded values in X, let $X \sqcup Y$ concatenate both collections, and set $\varepsilon = 10^{-12}$. To compare windows or runs with different scales, we used

$$D(X,Y) = \frac{|\mu(X) - \mu(Y)|}{\max\{\sigma(X \sqcup Y),\ 0.05|\mu(X \sqcup Y)|,\ \varepsilon\}}. \tag{2}$$

For a local event compared with a neighboring baseline B, we used

$$H(X,B) = \frac{|\mu(X) - \mu(B)|}{\max\{\sigma(X \sqcup B),\ 0.05|\mu(B)|,\ \varepsilon\}}. \tag{3}$$

The windows were $W = [325, 375]$, $L = [255, 315]$, $R = [385, 445]$, and the later TR2 window $W' = [675, 725]$. Write $X_r(W)$ for the values recorded in window W of run r.

C1: a within-run change across replay states. Compare TR2 near steps 350 and 700:

$$C_1 = D\big(X_{\mathrm{TR2}}(W), X_{\mathrm{TR2}}(W')\big) \geq 0.5. \tag{4}$$

The early window is within the first traversal; the later one covers the onset of replay. This rule retains curves that differ across these states.

C2: a cross-run difference at the same training time. Compare TR1 and TR2 near step 350:

$$C_2 = D\big(X_{\mathrm{TR1}}(W), X_{\mathrm{TR2}}(W)\big) \geq 0.5. \tag{5}$$

The runs have the same elapsed optimizer steps and learning-rate schedule, but only TR1 has entered replay.

C3: a boundary event stronger than random repetition. For each run, define

$$E_r = H\big(X_r(W), X_r(L) \sqcup X_r(R)\big). \tag{6}$$

Require

$$C_3 = \max(0, E_{\mathrm{TR1}} - E_{\mathrm{random}}) \geq 0.5. \tag{7}$$

This retains candidates whose local TR1 event exceeds the change at the same time under random repetition.

C4: agreement with the imposed replay period. Within the first 700 steps of the 50/50 run, pair all recorded values separated by exactly 100 steps. Let U, V be the paired columns and ρ_{100} their Pearson correlation. Define

$$a = \frac{\min\{\sigma(U), \sigma(V)\}}{\max\{\sigma(U \sqcup V), \varepsilon\}}, \qquad C_4 = \max(0, \rho_{100}) \min(1, a) \geq 0.15. \tag{8}$$

Only pairs with $t \geq 1$ and $t + 100 \leq 700$ are used. The score is zero when fewer than ten pairs are available.

C5: a center window distinct from both neighbors. Require the TR1 center to differ from both the preceding and following windows:

$$C_5 = \min \begin{Bmatrix} H\big(X_{\mathrm{TR1}}(W), X_{\mathrm{TR1}}(L)\big), \\ H\big(X_{\mathrm{TR1}}(W), X_{\mathrm{TR1}}(R)\big) \end{Bmatrix} \geq 0.5. \tag{9}$$

Taking the smaller deviation rejects curves that differ on only one side.

A candidate had to pass all five thresholds. For manual-review ordering, the script saturated each score as $q_k = C_k/(1 + C_k)$ and ranked candidates by

$$S = 100 \left[0.7 \min_k q_k + 0.3 \ \mathrm{mean}_k \ q_k \right]. \tag{10}$$

The windows and thresholds were exploratory heuristics. These scores narrowed the set for inspection; they were not significance tests or causal evidence.

B.2.3 Manual inspection and component removal

Curves that were identical across the four runs, or differed only by a fixed scale factor, were merged. Representative statistics were chosen for clear parameter-group attribution and interpretability. Manual inspection of the remaining small set highlighted the n-gram table's update magnitude and update–parameter direction relationship. The update magnitude rose after replay began, shifted later in TR2, and followed the alternation between new and replayed data in the 50/50 run (Figure 12). Code inspection confirmed that `ngram_embeddings` referred to lookup parameters in `bigram_ves` and `trigram_ves`. The n-gram pathway therefore became the main candidate for component intervention.

We then trained the full model and a model with the bigram/trigram value pathways disabled under the same single-shard sequential-replay setting, 1000-step budget, and seed. Unigram value embeddings were retained. The full model forked sharply after replay began. Disabling the two higher-order pathways removed this pattern, and validation BPB continued to improve in later epochs. Final validation BPB was 1.6769 for the full model and 1.0768 for the intervention. This supports a causal role for the n-gram pathway in this configuration. The disappearing feature was the rapid replay-aligned divergence, not every train–validation difference.

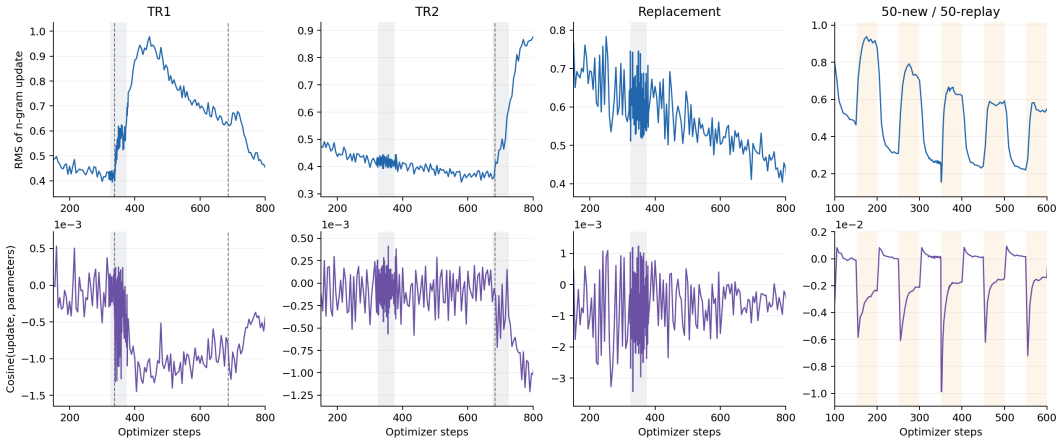


Figure 12: Two representative n-gram observables. The top row shows the RMS magnitude of the RMSProp update vector; the bottom row shows its cosine similarity with the parameter vector. Columns correspond to the four controls. Shading and boundaries follow Figure 11. Each panel uses its own visible vertical range. Unsmoothed curves are used to compare event timing and shape.

C Standard Setting and Measurement Conventions

C.1 Loss and frequency measurements

The discovery-stage configuration in Appendix B.1 and the controlled setting in Section 2 should be distinguished. They use different optimizers and injection implementations. Frequencies $F(i)$ refer to counts in a single unrepeated training corpus, even when that corpus is consumed many times.

For online curves, the training loss is obtained from the training forward pass before the corresponding update, while validation can be evaluated after that update. A fixed-probe gap instead compares fixed training and validation batches evaluated at the same checkpoint:

$$G_{\text{probe}}(t) = L_{\text{val,probe}}(t) - L_{\text{train,probe}}(t). \tag{11}$$

Where a matched no-gram control is subtracted, we explicitly use

$$G_{\text{net}}(t) = G_{\text{ngram}}(t) - G_{\text{nogram}}(t). \tag{12}$$

Cross-entropy values are in nats unless stated otherwise. BPB is the original search metric and should not be numerically equated with token cross-entropy.

> **Draft note.** Before submission, complete the main-run manifest with exact data and tokenizer fingerprints, table sizes for each capacity condition, batch and probe definitions, checkpoint alignment, and the fitting interval and weighting for each quantitative claim. The source manuscript does not yet provide all of these fields.

C.2 Numerical precision and implementation

The main setting uses BF16 autocast for forward computation. Model weights, n-gram table parameters, and optimizer states remain in FP32; cross-entropy and statistical aggregation are also computed in FP32. Diagnostic forwards use the same BF16 convention as the main evaluation, with per-token losses returned to FP32. Asynchronous aggregation changes execution, not the metric definitions.

FP8 was not used as the baseline. In a setting-validation test, simply selecting `float8_e4m3fn` in CUDA autocast did not work on the current eager `nn.Linear`/`addmm` path. FP8 training requires a dedicated implementation, such as scaled matrix multiplication or Transformer Engine, together with explicit scaling policies for activations, weights, and gradients. Precision boundaries would also have to be specified separately for lookup tables, attention, MLPs, logit projection, table optimization, and diagnostics.

A historical matched-hyperparameter comparison on an H200 found BF16 approximately 5.3 times faster per step than FP32. At 1000 steps, the reported raw gaps were 0.825 for BF16 and 0.858 for FP32, with similar trajectories. This check supported the conclusion that the numerical format did not remove forking in that setting; it is not a substitute for the evidence from the current main runs. The default convention remains BF16 forward computation, FP32 weights and optimizer states, and no `torch.compile`. Any future FP8 comparison should report stability, loss, gap, margins, throughput, and memory use separately.

D Full Intervention and Replay Curves

Figure 13 and Table 2 retain the main intervention evidence from the Chinese draft. Each intervention addresses a different part of the proposed pathway: context-dependent reads, stable hash assignment, table writing, or backbone adaptation. Interpretations are restricted to the tested thresholds and measurement windows.

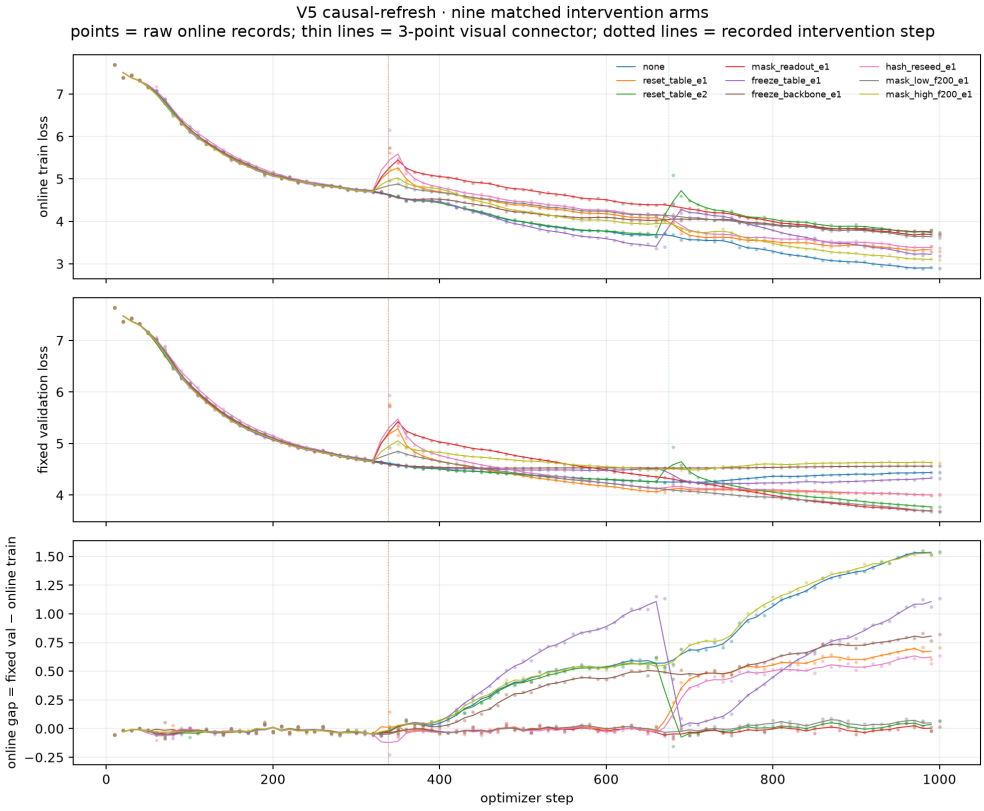


Figure 13: Training, validation, and gap trajectories for the component interventions. The hash-reseeding condition changes the context-to-row mapping at each epoch while retaining table weights and optimizer state. Its reported endpoint raw gap is 0.069.

Figure pending. The source high-frequency scan uses the old strict condition $F > t$. It is preserved as an asset but must be replaced by the intended $F \geq t$ scan before a threshold-matched comparison is presented.

Figure 14: High-frequency masking: replacement plot pending under the inclusive threshold convention.

The source analysis reports approximately linear aggregate growth over a 20-pass window, with a slope of 0.664 per pass and $R^2 = 0.998$. The 20-pass trajectories appear in the right panels of Figure 5; Figure 15 retains a separate, earlier 10-pass comparison. Boundary jumps account

Table 2: Intervention summary across comparisons in the current draft, not a list of endpoints for Figure 13. Raw gaps and relative reductions have different denominators. Exact run and checkpoint metadata for these source-reported numbers remain to be completed.

Condition	Intervention	Reported observation
No-gram	Remove the n-gram module	Raw gap 0.233; ordinary train–validation differences remain.
N-gram	Input injection; table LR $128\times$	Raw gap 5.676, with replay-related steps.
Low-frequency mask	Mask low-count reads in training and evaluation	$F \leq 8$ removes about 72% of net gap; higher thresholds further reduce it.
High-frequency mask	Mask high-count reads	Much smaller change than low-frequency masking in the tested comparison.
Hash reseed	Change context-to-row mapping each epoch; retain parameters and optimizer	Raw gap 0.069; substantial suppression of replay accumulation.
Freeze table	Stop table updates from epoch 2	About 94% of second-pass gap remains; growth returns in epoch 3, at a lower level than the two-pass reference.
Freeze backbone	Stop backbone updates from epoch 2	Slightly smaller second-pass gap; little further growth in epoch 3.

for approximately 28% of the accumulated gap in the source analysis, and their share decreases over passes. This motivates a dynamical account that includes within-pass changes, rather than attributing all damage to an instantaneous epoch transition. A finite-window linear fit does not establish unbounded long-run growth.

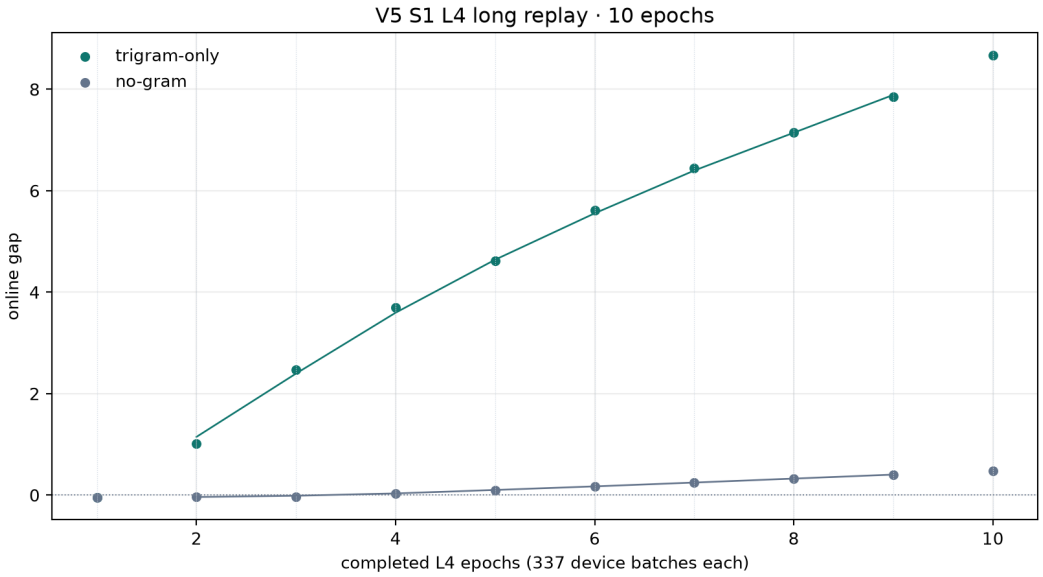


Figure 15: Archived 10-epoch trigram-only replay comparison with a no-gram control. The Chinese source caption describes 20 epochs, but the embedded image covers ten. The image is retained here with its actual scope; it is not the evidence for the 20-pass fit quoted in the text.

A reduced gap must be distinguished from improved validation performance: increasing training loss can shrink the difference without repairing generalization. Likewise, a gap that survives table freezing does not imply that all relevant memory is stored in the table. In the strong-replay comparison recorded in the source, validation loss is approximately 6.95 with n-gram memory and 3.34 without it. Figure 16 compares absolute validation changes relative to the no-gram baseline. In this regime, the additional memory does not provide a net generalization benefit.

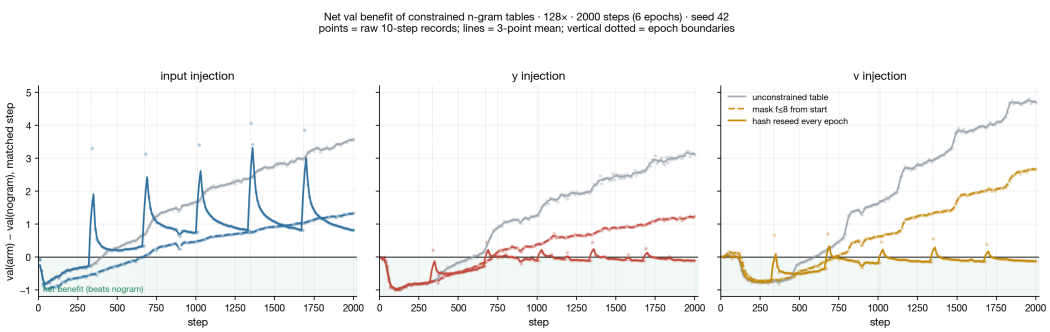


Figure 16: Validation-loss differences relative to the no-gram baseline in the source intervention analysis. These comparisons delimit the benefit of memory in the measured replay regime; they do not establish that n-gram memory should only be used as a short-term cache.

Low-frequency masking and reseeding are mechanism-motivated interventions, but their practical value also depends on preserved training benefits and additional cost. The choice between refreshing memory, retaining it, or protecting selected keys requires evidence under the intended data coverage and training history.

E Mathematical Derivations and Model Tests

E.1 From novel continuations to the frequency law

Fix a training step t and group prediction positions by the training count $F \geq 1$ of their context. Within a frequency bin, let $f(F)$ be the fraction of validation positions whose true continuation never followed that context in training. The token itself may be common; what training never showed is the context–continuation pair. The gap splits exactly into two terms:

$$\begin{aligned} \mathrm{gap}_t(F) &= f(F)D_t(F) + \big(1 - f(F)\big)S_t(F), \\ D_t(F) &= \ell^{\mathrm{val}}_{\mathrm{novel},t}(F) - \ell^{\mathrm{tr}}_t(F), \qquad S_t(F) = \ell^{\mathrm{val}}_{\mathrm{seen},t}(F) - \ell^{\mathrm{tr}}_t(F). \end{aligned} \tag{13}$$

Here D_t and S_t compare novel and seen validation losses with the same online training loss. All three losses are bin means at the same logged step, rather than losses matched context by context. Coverage sets the mixture weights; the model determines the two loss differences.

If both differences vary little with F, this decomposition is approximately affine in the novel fraction:

$$\mathrm{gap}_t(F) \approx a_t f(F) + b_t. \tag{14}$$

When the seen term is also negligible, it reduces to $\mathrm{gap}_t(F) \approx D_t f(F)$: the gap is the fraction exposed to crowding-out times its loss penalty. The affine relation describes the measured bins well (Figure 17b,c), but the fitted intercept is not generally the seen term; frequency-dependent components can also produce a close fit.

Good–Turing connects missing continuation mass to singleton counts (Appendix E.2). For a fixed context with a conditional tail $p_{(r)} \sim Ar^{-s}$, the expected missing mass decays as $F^{-(1-1/s)}$ over an intermediate range (Equation (20)). This gives a source of slow, approximately power-law decay. Pooling heterogeneous contexts and including the seen term can change the gap's slope, so coverage helps explain the shape without fixing a universal exponent. Appendix E.5 tests the approximation and its limits.

E.2 Good–Turing estimation and continuation coverage

Good–Turing estimation relates unseen probability mass to observed singletons (Good, 1953; Gale & Sampson, 1995). For context i, let p_{iy} be its continuation distribution and $m_i(F) = \sum_y p_{iy}(1-p_{iy})^F$ its expected missing mass after F independent draws. If $K_{i,1}(k)$ counts continuation types occurring

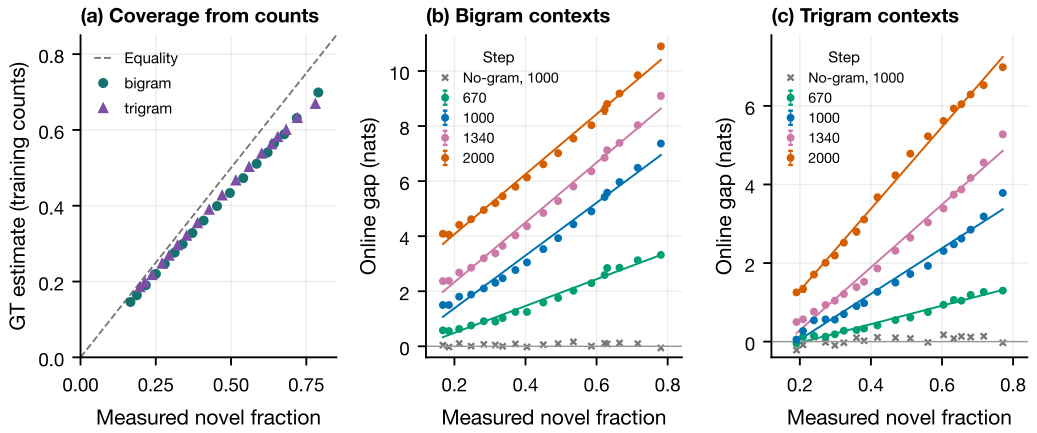


Figure 17: Coverage and the gap. (a) Novel fraction predicted from training counts against directly measured held-out novelty; the dashed diagonal is equality. (b,c) Online gap against the directly measured novel fraction, with affine fits at four training steps. Each point is a frequency bin; bars in (b,c) show the standard deviation across three seeds. The no-gram control is shown at step 1000. All panels use linear axes; protocol and residuals are in Appendix E.5.

exactly once in k draws, then

$$\frac{\mathbb{E}K_{i,1}(F+1)}{F+1} = \sum_y p_{iy}(1-p_{iy})^F = m_i(F). \tag{15}$$

This is an expectation identity for a fixed context. By contrast, $K_{i,1}(F)/F$ has expectation $m_i(F-1)$ and equals one at $F=1$ by construction. Neither statement by itself identifies the validation-token-weighted missing mass across different contexts observed F times.

Heterogeneous contexts and token weighting. Assume training context counts N_i are Poisson with means $\lambda_i = Tq_i$, continuations are conditionally independent, and an independent validation token has context distribution q_i and the same conditional probabilities p_{iy}. Write $\pi_F(\lambda) = e^{-\lambda}\lambda^F/F!$. Conditional on the validation context having training count F, its population novelty probability is

$$f_\star(F) = \frac{\sum_i q_i \pi_F(\lambda_i) m_i(F)}{\sum_i q_i \pi_F(\lambda_i)}. \tag{16}$$

Define $n(F) = \sum_i \mathbf{1}[N_i = F]$ and $N_1(F) = \sum_i \mathbf{1}[N_i = F]K_{i,1}(F)$. Using $(F+1)\pi_{F+1}(\lambda) = \lambda\pi_F(\lambda)$ gives

$$\begin{aligned} \mathbb{E}N_1(F+1) &= T\sum_i q_i \pi_F(\lambda_i) m_i(F), \\ (F+1)\mathbb{E}n(F+1) &= T\sum_i q_i \pi_F(\lambda_i), \\ f_\star(F) &= \frac{\mathbb{E}N_1(F+1)}{(F+1)\mathbb{E}n(F+1)}. \end{aligned} \tag{17}$$

This is a ratio of expectations under the sampling model, not an equality for every realized corpus or an unbiasedness guarantee for the empirical ratio. For a bin B of exact counts, the corresponding train-only estimator is

$$\widehat{f}_{\mathrm{GT}}(B) = \frac{\sum_{F\in B} N_1(F+1)}{\sum_{F\in B}(F+1)n(F+1)}. \tag{18}$$

Its pooling weights use training counts. Weighting exact-layer estimates by held-out token counts would use validation information.

Uncalibrated corpus check. Counts use training shard 1 (49,716,936 tokens); the full held-out check uses 49,102,848 prediction positions from shard 2. Context counts follow the recorded raw-concatenation frequency index, while validation keys follow the model's 2049-token chunk layout. Natural text need not satisfy the Poisson or conditional-independence assumptions. In the 18 native bins over $F = 1$–1000, Equation (18) gives 0.870–0.885 times the directly measured novel fraction for bigram contexts and 0.857–0.937 for trigram contexts (Figure 17a). At $F = 1$, measured novelty is 0.790 and 0.780, compared with shifted estimates 0.699 and 0.669. No validation-fitted scaling factor is applied. A similar underestimate appears in within-training-shard splits, suggesting that train–validation domain shift alone is insufficient to explain it; these observational checks do not identify document repetition as its cause.

Conditional tails. For a context c_i with $F_i = F$, let $y_{i,(r)}$ denote the r-th most likely continuation and $p_{i,(r)}$ its true conditional probability. Model the observed continuations $Y_{i,1}, \ldots, Y_{i,F}$ as independent draws, with counts $n_{i,(r)}(F) = \#\{k \in \{1, \ldots, F\} : Y_{i,k} = y_{i,(r)}\}$. The novel-continuation rate $f_i(F)$ and its expectation under this sampling model are

$$f_i(F) = \sum_r p_{i,(r)} \mathbf{1}\left[n_{i,(r)}(F) = 0\right], \quad \mathbb{E}[f_i(F)] = \sum_r p_{i,(r)}(1 - p_{i,(r)})^F. \tag{19}$$

Assume a conditional tail $p_{i,(r)} \sim A_i r^{-s}$ with $s > 1$. For large F within the tail's scaling range, before finite-vocabulary effects become significant, the dominant probabilities are of order $1/F$. Approximating $(1 - p_{i,(r)})^F$ by $e^{-F p_{i,(r)}}$ and the rank sum by an integral then gives (Gnedin et al., 2007)

$$\begin{aligned} \mathbb{E}[f_i(F)] &\sim \int_1^\infty A_i r^{-s} e^{-F A_i r^{-s}} \, \mathrm{d}r \\ &\sim \frac{A_i^{1/s}}{s} \Gamma\left(1 - \frac{1}{s}\right) F^{-(1-1/s)} \\ &\propto F^{-\alpha}, \quad \alpha = 1 - \frac{1}{s}. \end{aligned} \tag{20}$$

This gives an asymptotic account of slow missing-mass decay for a fixed context; it does not determine the cross-context gap exponent of Section 3.1. Finite vocabulary limits the power-law regime to an intermediate range. Heterogeneity across contexts may alter the observed effective exponent.

E.3 Crowding-out within a context subspace

Fix a context c_i and call the next-token logits computed from it, a_v for v in the vocabulary, its *context subspace*; the n-gram table gives c_i a learned coordinate, which can make its logits respond differently from those of other contexts (Section 4.2). Partition the vocabulary into the continuations of c_i seen in training ($\mathcal{S}$), those that occur only in validation ($\mathcal{V}$), and the rest. With $p_v = \mathrm{softmax}\, a_v$, one idealized gradient step directly in logit coordinates on a training target $C \in \mathcal{S}$ moves every logit by

$$a_v^{(t+1)} - a_v^{(t)} = -\eta \frac{\partial \ell_C}{\partial a_v} = -\eta \left(p_v^{(t)} - \mathbf{1}[v = C]\right), \qquad \eta > 0, \tag{21}$$

raising a_C by $\eta(1 - p_C)$ and lowering every other logit by ηp_v. For any $u \notin \mathcal{S}$ the odds $z(t) = p_C^{(t)}/p_u^{(t)}$ therefore satisfy

$$z(t+1) = z(t) \exp\left[\eta \left(1 - p_C^{(t)} + p_u^{(t)}\right)\right] > z(t), \tag{22}$$

For this one distribution, $\ell_u - \ell_C = \ln z$, so the loss difference between an unseen alternative and the observed target grows. This does not imply equal and opposite changes in their absolute losses, or identify $\ln z$ with the bin-averaged D_t in Equation (13). In a parameterized network a small gradient step instead gives $\Delta a \simeq -\eta J J^\top (p - e_C)$, where J is the logit Jacobian; optimizer preconditioning changes this coupling further. Equations (21)–(22) therefore describe an idealized sharpening mechanism, rather than a derivation of the measured coefficients.

E.4 POSITION-CLASS DECOMPOSITION OF THE EXCESS VALIDATION LOSS

Table 3 decomposes the both-table model's excess validation loss over the no-gram control (runs `s1v5_128_marm_both_ckpt_fixed` and `s1v5_128_marm_nogram_ckpt_fixed`, seed 42, step 1000) by whether the true continuation followed its context in the training shard. Contexts are counted at bigram or trigram granularity; *novel* includes positions whose context itself is absent from training.

Table 3: Share of the step-1000 excess validation loss (1.68 nats per token, 589,248 positions) carried by each position class.

context	class	share of positions	share of excess loss
bigram	novel continuation	31%	73%
bigram	novel or seen with $F \leq 31$	39%	77%
trigram	novel continuation	66%	100%

These are shares of the excess *validation* loss relative to no-gram, not shares of a model's train–validation gap. They do not imply $S_t(F) \approx 0$. In particular, the table includes $F = 0$ positions; the frequency-gap tests below exclude them because no training counterpart exists.

E.5 TESTS OF THE COVERAGE APPROXIMATION

Protocol. The online-gap tests use `fig1v5_rp_both_s42_fixed`, `fig1v5_rp_both_s43_fixed` and `fig1v5_rp_both_s44_fixed`, with matched `fig1v5_rp_nogram_s42_fixed`, `fig1v5_rp_nogram_s43_fixed` and `fig1v5_rp_nogram_s44_fixed` controls. They use the standard input-injection setting: RMSProp tables without momentum, table LR $128\times$, 2^{20} rows per table, backbone LR 0.0006, and a 100-step warmup followed by constant LR. Training replays shard 1; the fixed validation pool contains 589,824 positions from the first four batches of shard 2. Direct novelty counts match every plotted validation bin's token count exactly. Training losses are recorded before the update and validation losses after it, at the same logged step.

All 18 native bins contained in $1 \leq F \leq 1000$ have at least 300 train and validation tokens at the reported steps; no positive-gap filtering is applied. The bins are $1, 2, 3, 4, 5$, 6–8, 9–12, 13–20, 21–30, 31–50, 51–75, 76–100, 101–150, 151–200, 201–300, 301–500, 501–750, 751–1000. Steps 670, 1000, 1340 and 2000 lie near the ends of passes 2, 3, 4 and 6, but are not exact pass boundaries. We fit the seed-mean gap by equal-bin least squares. The same held-out corpus supplies f for all seeds.

Fit quality. For fits to the three-seed mean at the four reported steps, relative rms residuals are 3–8% for bigram and 3–11% for trigram contexts (Table 4). The intercept matters: at step 2000 the bigram fit has $b_t = 1.89$ nats; forcing the line through zero increases rms error from 0.20 to 0.76 nats. Thus the proportional approximation does not describe the later gap as well as the affine relation.

Table 4: Online gap fit $af + b$ on 18 bins. Coefficients and rms errors are in nats; relative rms is divided by mean absolute bin gap. The holdout fit uses alternating frequency bins and evaluates the others, testing interpolation on this corpus rather than corpus transfer.

Context	Step	a	b	rms	Relative	Holdout
Bigram	670	4.87	−0.48	0.137	8.2%	0.169
	1000	9.58	−0.53	0.233	6.3%	0.269
	1340	10.86	0.16	0.205	4.1%	0.228
	2000	10.90	1.89	0.202	3.0%	0.233
Trigram	670	2.35	−0.50	0.063	10.7%	0.060
	1000	5.81	−1.11	0.155	9.9%	0.132
	1340	8.00	−1.31	0.172	7.2%	0.180
	2000	10.42	−0.79	0.129	3.2%	0.132

Validation-token weighting changes relative rms to 3.3–10.5% (bigram) and 3.2–9.1% (trigram). Residuals are structured and exceed seed variation; three training seeds do not quantify uncertainty

across corpora. The fit is a compact description, not a prediction of its own coefficients. Moreover, a close affine relation need not imply constant D_t or S_t: in the separate seed-42 step-1000 fixed-probe diagnostic, the bigram fit has a negative intercept while all measured seen terms are positive. Frequency-dependent terms can compensate.

Exact-count comparisons. The stored `s1v5_128_marm_both_ckpt_fixed` step-1000 probes (seed 42) allow a supplementary comparison at each exact $F = 2, \ldots, 8$. Contexts are stratified by their training singleton fraction into $[0, .25)$, $[.25, .5)$, $[.5, .75)$ and $[.75, 1]$, retaining cells with at least 200 train and validation positions. At bigram $F = 4$, the extreme strata have $f = 0.247$ versus 0.795 and diagnostic gaps 3.234 versus 6.013 nats. At trigram $F = 4$, gaps rise from 1.448 to 2.619 nats and then decline to 2.505 in the highest stratum. Thus the association goes beyond count alone but is not universally monotone. Singleton fraction also reflects conditional entropy and concentration; it is not randomized. These losses use fixed probes and probabilities floored at 5.96×10^{-8} (up to 1,094 zero probabilities in an array), so they are censored diagnostics distinct from the primary online gap. Novelty counts are unaffected by the loss floor.

What an exponent can mean. Even if $f(F) = cF^{-\alpha}$ exactly and $\mathrm{gap} = af + b$, the local gap exponent would be

$$\beta_{\mathrm{eff}}(F) = -\frac{\mathrm{d}\log \mathrm{gap}}{\mathrm{d}\log F} = \alpha\,\frac{af(F)}{af(F) + b}. \tag{23}$$

Here a and b are held fixed as F varies, and the logarithm requires a positive gap. The exponent equals α only when the intercept is negligible. Changes in coefficients with training or table capacity can change an observed exponent without changing the corpus. The registered table-size sweep and the time-dependent fits therefore preclude a corpus-fixed gap exponent. The evidence supports coverage as a contributor to the frequency dependence under the measured protocol; it does not establish a universal affine law across corpora, optimizers or injection paths, or coverage as the sole causal mediator.

F Mechanistic Experiments and Diagnostics

F.1 Sharpening diagnostics

Section 4.1 compares a context-only probe of "walk their dog," whose training continuation is *down* and whose fixed-validation continuations include *daily* and *for*, across replay passes (Figure 7a). The aggregate diagnostic separates validation positions with a training-seen bigram context by whether their true continuation was also seen after that context in training, then compares the both-table model with the no-gram control (Figure 7b). Predictive entropy and cross-entropy are computed on the same fixed validation positions so that sharper distributions can be distinguished from improved accuracy.

For the displayed context, *down* occurs once after the trigram in training; *daily* and *for* occur once each in the fixed validation window and not after this trigram in training. Panel (a) uses the seed-42 `sharpening_f99_contextonly_20260921_s42_fixed` context probe. At the ends of five passes, steps 337, 674, 1011, 1348, and 1685, it presents only the literal context and reads its full-vocabulary next-token distribution. Its training loss is $-\log p(\textit{down})$; the validation loss is the mean of $-\log p(\textit{daily})$ and $-\log p(\textit{for})$. Neither is the original full-prefix online loss. This one example illustrates crowding-out; it is not a claim that every context sharpens monotonically.

Panel (b) instead uses full causal prefixes and position-weighted validation cross-entropy. The archived aggregation `fig6_global_seen_novel_20260922_s42_fixed` plots the original both-table checkpoints and a separately evaluated no-gram control at the same five epoch ends. Of the 589,248 classified validation positions, 405,058 have a training-seen continuation and 158,790 a novel continuation after a training-seen bigram context; 25,400 contexts absent from training are excluded from these two curves. The entropy comparison also concerns fixed validation positions, not the context-only illustration. The two panels have different inputs and estimands and should not be read as a matched per-example loss decomposition.

F.2 CONTEXT-LOSS MASKING

The experiment in Section 4.2 excludes next-token loss for a random 2% of training trigram contexts beginning at epoch 2 or 3, while retaining their input tokens, trainable parameters, and the loss denominator. A disjoint 2% of unmasked contexts and a normal-training run from the same checkpoint are controls. Figure 8a compares the masked and peer sets using $C_S = \sum_{i \in S} q_i[\bar{\ell}_{\mathrm{val},i} - \bar{\ell}_{\mathrm{tr},i}]$ on fixed batches. Figure 8b tracks the probability assigned to each context's training-seen continuations in both the n-gram model and the no-gram control. The contrast tests whether removing a context's own updates stops its gap growth despite continued optimization on other contexts; it does not assume mathematically independent parameter subspaces.

An independently archived one-pass audit makes the cohort and measurement choices explicit. It samples 2% of exact training trigram contexts uniformly without replacement (selection seed 20260919); among 379,789 selected contexts, 5,565 have fixed-validation support (8,083 positions). A disjoint unmasked peer sample has 5,571 measurable contexts (8,082 positions), not the entire unmasked complement. In that audit, $q_i = N_{\mathrm{val},i}/589{,}824$ uses the original fixed-validation position count without renormalizing within either cohort. Per-context train and validation losses preserve their full causal prefixes at a common checkpoint, unlike the context-only sharpening probe. The epoch-2 pair is `ctxloss_e2_all_p02_control_s42_fixed` and `ctxloss_e2_all_p02_mask_s42_fixed` over steps 337–674; the separate epoch-3 pair is `ctxloss_all_p02_control_s42_fixed` and `ctxloss_all_p02_mask_s42_fixed` over steps 674–1011. Within each pair, normal and masked training begin from the same checkpoint, but these are separate experiments and are not the four-epoch trajectories displayed in Figure 8a. Suppressed aggregate growth on selected contexts alongside continued growth on peers supports approximate locality of additional gap accumulation. Individual losses still move in both directions, and a pre-existing gap can persist.

> **Draft note.** Before submission, register the entropy probe and its aggregation script, and identify the original run IDs and plotted-value manifest for the multi-epoch context-loss-masking and seen-mass panels in Figure 8. The separate one-pass audit above is not their figure provenance.

G CROSS-ARCHITECTURE REPLICATION AND INTERVENTION PROTOCOL

Section 2 introduces the cross-architecture replication; this appendix gives its protocol. The DeepSeek-like configuration is a smaller 12-layer model whose backbone combines MLA-lite attention, a dense SwiGLU first layer, 11 subsequent mixture-of-experts layers with 16 routed experts, top-2 routing and two shared experts, auxiliary-loss-free load balancing, and four-stream mHC. Engram modules at layers 2 and 6 use bigram and trigram retrieval with multi-head hashing, context-dependent gates, and causal local convolution, i.e. multi-layer input over-encoding in the formulation of this paper. It is trained for 2000 updates with seed 42, replaying the fixed shard-1 corpus with a fixed validation probe; the backbone uses AdamW with learning rate 0.0006 and the tables RMSProp with learning rate 0.0768, with 100 warmup steps followed by constant learning rates. Figure 18 compares the current training batch before each logged update with fixed validation batches after that update, not two fixed probes at one checkpoint. Table 5 identifies the three independent runs used in that figure.

Table 5: Aligned DeepSeek-like runs in Figure 18. All use seed 42; losses are in nats at step 2000.

Condition	Run ID	Train	Val.	Gap
Full repeat	`dsalign20260921_control_s42_2k_fixed`	1.689	5.005	3.316
Mask $F \leq 8$	`dsalign20260921_maskle8_s42_2k_fixed`	2.104	3.938	1.835
Mask $F \leq 100$	`dsalign20260921_maskle100_s42_2k_fixed`	2.588	3.405	0.817

The low-frequency conditions suppress bigram and trigram retrieved vectors beginning at update 338, for raw input contexts with training frequency $F \leq 8$ or $F \leq 100$ (inclusive). The masks act after

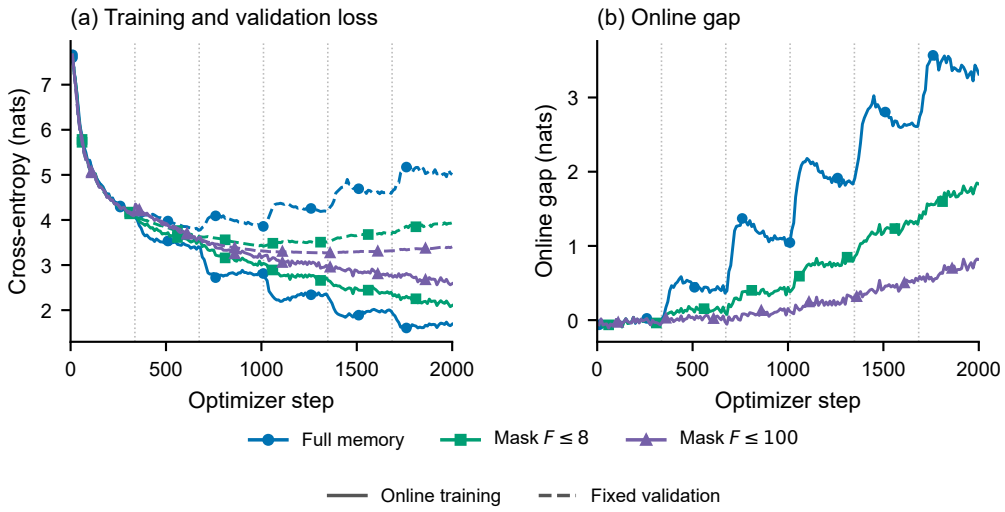


Figure 18: Forking and low-frequency retrieval masking in the DeepSeek-like model. (Left) Pre-update online training loss (solid) and post-update fixed validation loss (dashed). (Right) Their difference at each logged step. Colors and markers identify the unmasked model and two masking thresholds. Vertical dotted lines mark replay boundaries; line segments connect raw logged values without smoothing.

retrieval and before the key/value projections in training and evaluation; all next-token loss terms remain. Counts come from the unrepeated training frequency index, whose SHA256 fingerprint is `ed5f29fc14a3bc48ba196143e859c3613e6032c849ab81533b85682e3043a986`. Table parameters, optimizer state, and remaining fusion operations are retained. Each run yields 200 unsmoothed logged values, at steps 10 through 2000, and all three aligned traces are finite. The runs have separate training trajectories and were not resumed from one shared pre-intervention checkpoint. Their agreement is therefore a same-seed, configuration-level comparison, not a bitwise-matched checkpoint intervention or a multi-seed uncertainty estimate. A stable matched no-memory control remains unavailable.

In a separate historical all-read-mask run, the first failure report records an Inf at step 96 on the input boundary of `blocks.8.ffn.shared_experts.0.down.0`. That run completed 95 finite updates; it is not in Figure 18 or Table 5, and is not a successful no-memory baseline.

A further mechanism stress test could select contexts with several valid continuations, repeatedly train one, and measure the losses of the others. This asks whether local prediction becomes excessively sharp, rather than merely testing a broad distribution shift. Its feasibility and scope require matched experiments. Neither the current replication nor this proposal establishes a vulnerability in a deployed DeepSeek model.

> **Draft note.** Before submission, register the complete architecture and data/tokenizer fingerprints alongside the aligned figure manifest, document all differences from the minimal setting, and obtain a stable matched no-memory control if a stronger component-specific claim is desired.

H Continued-Training and Replay Stress-Test Protocol

The six runs in Section 5 start from the checkpoint-materialization rerun of the full-pool $20\times$ trigram experiment, with endpoint gap 0.214 at step 20,220. The original scientific run has the same setting and endpoint gap 0.220 but does not retain final weights. We restart the backbone AdamW and table RMSProp states and apply a fresh 100-step warmup.

The model is the vanilla 8-layer, 6-head, 768-dimensional NanoGPT with input trigram injection and a clean 2^{20}-row table. We use seed 42, BF16 forward computation, no compilation, backbone learning rate 6×10^{-4}, table learning rate 0.0768, and fixed validation. Each continued-pretraining source has 333 batches and is run for five epochs (1665 updates); logs are recorded every 10 steps and at the final boundary. The online gap is $G = L_{\mathrm{va}} - L_{\mathrm{tr}}$, where training loss is measured on the current batch before the update and validation loss is measured after the update on fixed batches.

The endpoint measurements are:

Table 6: Endpoint measurements for continued language-model pretraining from the low-gap checkpoint. All values are nats, seed 42, step 1665. The table-trainable and table-frozen runs differ only in whether the trigram lookup table receives gradients.

Source	Table	Train	Validation	Gap
ID	Trainable	1.913	4.376	2.463
ID	Frozen	3.030	3.372	0.341
Magicoder	Trainable	0.740	1.752	1.012
Magicoder	Frozen	1.342	1.555	0.212
OpenR1	Trainable	0.754	1.480	0.726
OpenR1	Frozen	1.226	1.324	0.098

The corresponding dense logs and figures are archived with run IDs `sft_*_e5_freq10_{trainable,frozen}_s42`. These are single-seed continued-pretraining measurements, not response-only instruction-SFT results.

Static RLVR-like proxy. We additionally ran two continued-pretraining runs on `allenai/RLVR-MATH` from the Tulu 3 release (Lambert et al., 2025) (revision `bd2a93551b503a395fadd1a740d957559cfe6f3c`), which contains the 7,500 MATH training problems formatted for RLVR. The prompts carry fixed 4-shot exemplar prefixes that are byte-identical across examples, and the `ground_truth` field is the reference final answer. The original parquet SHA256 is `2fb8483bf7a4ae9821bee1e715490fb8de5b59f0d2b58bd4de78cc75094d9422`. Messages were retained verbatim and the `ground_truth` was appended once after an "Answer:" marker. The first 95% and last 5% of source examples (in original row order) were used as disjoint train and validation pools, then cycled to 333 train batches and 4 validation batches.

The shared setting matches the six runs above: seed 42, 1665 updates, five replay epochs, fresh optimizer state and 100-step warmup, input trigram-only clean 2^{20}-row table, and fixed validation. In the frozen arm only the table is fixed; the backbone remains trainable. Endpoint measurements are:

Table 7: Static RLVR-like verifier-selected continued-pretraining endpoints. All values are nats at step 1665, seed 42.

Source	Table	Train	Validation	Gap
RLVR-MATH	Trainable	0.009	0.621	0.612
RLVR-MATH	Frozen	0.020	0.522	0.502

The run IDs and dense logs are archived under `rlvr_static_proxy_20260921`. This is a static verifier-selected proxy, not on-policy reinforcement learning.